

北京联合大学

毕 业 设 计

题目：基于 SSM 框架的车票售卖系统设计与实现



姓 名：李子轩

学 号：2022240332011

学 院：智慧城市学院

专 业：计算机科学与技术

指 导 教 师：戴庆龙

协助指导教师：

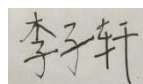
2026 年 5 月 2 日

北京联合大学

本科毕业设计诚信承诺书

本人慎重承诺和声明,在毕业设计活动中遵守学校有关规定,恪守学术规范,在本人的毕业设计内容除特别注明和引用外,均为本人观点,不存在剽窃、抄袭他人的学术观点、思想和成果,不存在伪造、篡改实验数据。如有违规行为发生我愿承担一切责任,接受学校的处理,并承担相应的法律责任。

学生(签名):



2026年 5月2日

摘 要

本论文围绕基于 SSM 框架的车票售卖系统设计与实现展开，目标是基于现有项目代码，完成一个前端、后端、数据库联动的完整 Web 系统设计与实现。系统前端采用 Vue3、Vue Router、Axios 和 Element Plus 组成交互层，后端采用 Spring MVC、Service、MyBatis 实现业务与数据访问分层，数据库使用 MySQL 8.0 存储用户、车次、席位库存和订单数据。

课题在实现中坚持“先打通，再优化”的技术路径。也就是说，先让用户从登录到下单再到退票这条主线完整跑通，之后再逐步补齐管理员统计、库存调节、异常提示和部署脚本。这样做的好处是风险可控，不会出现功能很多但系统跑不起来的情况。从最终效果看，这条路径是可行的，核心流程稳定，数据变化也符合预期。

论文重点分析了系统的需求拆解、模块划分、数据库设计、接口契约、鉴权机制和部署流程。针对订单业务中最容易出问题的“余票计算”场景，系统采用“基础库存 - 已售数量 + 管理调整”模型，并在下单环节加入事务控制，以降低并发写入时的数据不一致风险。针对接口安全，系统采用 JWT 无状态鉴权，减少服务端会话压力，也方便前后端分离架构下的统一认证。

测试结果表明，系统已能够完成注册登录、车次查询、订单创建、支付、取消、退票、管理员用户管理、订单查询、销量统计和库存增减等功能。本文最终形成了一套可复现、可演示、可继续扩展的本科毕业设计方案。

关键词：铁路车票系统 前后端分离 Spring MVC MyBatis MySQL JWT

Abstract

This thesis focuses on the design and implementation of a ticket selling system based on the SSM framework.

The development strategy follows a practical sequence: first make the end-to-end main flow available, then refine the details of data consistency, exception handling, and deployment automation. This strategy keeps the implementation controllable and avoids architecture that looks complete but cannot run stably.

In terms of backend business logic, remaining ticket calculation adopts a deterministic model based on base stock, sold quantity, and manual stock adjustment, while transactional processing is applied to critical order operations. For security, JWT is used for stateless authentication, which fits well with front-end and back-end separation.

The final system has passed multi-scenario functional testing and can support a complete demo-level ticket workflow. The output of this thesis is a reproducible and extensible engineering solution suitable for undergraduate graduation project requirements.

Key Words:Railway Ticket System Front-end/Back-end Separation Spring MVC MyBatis MySQL JWT

目 录

摘 要I

引 言 1

1 绪论 2

1.1 研究目的及意义 2

1.2 国内外研究现状 2

1.3 研究内容 2

1.4 论文组织结构 2

2 相关技术与可行性分析 4

2.1 核心技术说明 4

2.2 可行性分析 5

2.2.1 技术可行性 5

2.2.2 经济可行性 5

2.2.3 操作可行性 5

2.2.4 时间可行性 5

3 系统分析与设计 7

3.1 需求分析 7

3.1.1 功能需求 7

3.1.2 可行性分析 9

3.1.3 非功能需求 9

3.1.4 约束条件 10

3.1.5 异常处理设计 10

3.1.6 系统性能考虑 10

3.2 系统总体开发设计 11

3.2.1 系统总体架构	11
3.2.2 前端界面设计	12
3.2.3 后台系统设计	13
3.3 系统模型	14
3.3.1 系统动态模型	14
3.3.2 系统静态模型	15
3.4 系统数据库设计	15
3.4.1 系统实体及主要属性	15
3.4.2 系统 E-R 图	16
3.4.3 实体详细表	17
4 系统实现与部署	20
4.1 系统开发环境	20
4.2 系统部署流程	20
4.3 用户认证与权限控制模块实现	21
4.4 车次查询与余票计算模块实现	22
4.5 订单创建与事务控制模块实现	23
4.6 订单状态流转模块实现	23
4.7 管理员库存调整模块实现	24
4.8 统一响应与全局异常处理实现	24
4.9 前端路由与权限拦截实现	25
5 系统测试与结果分析	28
5.1 测试环境与工具	28
5.1.1 测试环境	28
5.2 功能测试	28

5.3 性能测试	30
5.4 测试结论	30
5.4.1 测试用户注册、登录与个人资料	30
5.2.2 测试车次查询与车次列表	33
5.2.3 测试购票下单（创建订单）	35
5.2.4 测试我的订单（支付、取消、退票、确认出行）	36
5.2.5 测试管理端统计与订单管理	37
5.2.6 测试管理端车次维护与库存调节	38
6 总结和展望	40
6.1 工作总结	40
6.2 展望	41
致 谢	43
参考文献	44

引 言

随着我国铁路交通运输行业的持续发展，高铁、普速列车网络覆盖日益广泛，铁路出行已成为民众跨区域出行的首选方式，铁路车票的线上售卖需求持续增长。传统车站人工售票模式效率低下，难以满足用户便捷购票的需求；而现有大型售票系统功能繁杂，对于中小型铁路运营场景或专项售票需求的适配性不足。因此，开发一款功能简洁、操作便捷、专注于核心售票业务的铁路车票售卖管理系统具有重要的现实意义。

SSM（Spring + Spring MVC + MyBatis）框架作为 Java 企业级开发的成熟框架，具备耦合度低、稳定性强、数据交互高效等优势；Vue 框架则以轻量灵活、前端交互友好、组件化开发便捷等特点，成为主流前端开发技术。本课题采用“SSM+Vue”前后端分离架构，开发一款专注于铁路车票售卖的管理系统，摒弃复杂的抢票功能，聚焦车票查询、余票校验、订单生成、基础管理等核心业务，打造操作简洁、响应迅速、适配中小型运营场景的铁路车票售卖解决方案。

针对上述问题，本文围绕铁路车票售卖业务场景完成需求分析与系统设计，提出并实现一套基于“SSM（Spring + Spring MVC + MyBatis）+ Vue”前后端分离架构的车票售卖管理系统。后端以 Spring Boot 承载接口服务与业务规则，MyBatis 完成数据库持久化访问与 SQL 映射；前端采用 Vue3 + Element Plus 构建页面交互，并通过 Axios 与后端 RESTful API（/api/user、/api/train、/api/order、/api/admin 等）对接，配合 JWT 完成登录态校验与权限控制。系统聚焦车次查询、余票校验与库存调节、订单生成与状态流转、用户端购票流程与管理端统计维护等核心功能，舍弃抢票等高复杂度模块，面向中小型运营或教学演示场景提供功能简洁、部署可行、联调路径清晰的实现方案，并为后续扩展缓存、监控与支付接入预留接口层设计空间。

1 绪论

1.1 研究目的及意义

线上票务服务已经成为常态化基础服务，用户对系统体验有两个最直接的感受：一个是能不能很快查到车票，另一个是下单以后状态是不是清晰。换句话说，用户不关心你用了多少技术名词，只关心流程顺不顺、结果准不准。

在本科毕业设计场景里，这个课题的价值很明确。它能把前端页面、后端接口、数据库模型、部署流程这些原本分散的知识点串成一条完整工程链路。通过这条链路，开发者可以从“会写某个功能”提升到“能做成一个系统”。

从教学实践角度看，铁路票务场景天然具备流程闭环，包含查询、下单、支付、状态变更、管理统计等多个典型业务节点，适合作为综合性项目。项目实现后，既可以用于答辩展示，也可以作为后续课程演示平台继续迭代。

1.2 国内外研究现状

Web 系统开发领域中，Spring 生态长期占据主流地位，特别是在中小型业务系统里，基于 Controller-Service-DAO（或 Mapper）的分层方式仍然具有较高可维护性。相关研究普遍认为，清晰分层能够降低模块耦合，便于团队协作和后期扩展。

在前端方向，Vue3 由于语法简洁、社区活跃、生态工具完善，已经成为教学和工程实践中非常常见的方案。与 React 或 Angular 相比，Vue3 对初学者更友好，项目搭建成本较低，适合毕业设计周期内快速形成可交付系统。

数据库方面，MySQL 依然是高频选择。其事务机制、索引优化能力和部署便利性，使其在票务、订单、用户等结构化业务中表现稳定。综合已有研究和实际项目经验，选用 Vue3 + Spring MVC + MyBatis + MySQL 作为本文技术路线具有充分合理性。

1.3 研究内容

本文首先梳理系统需求，明确普通用户端和管理员端的业务边界；其次完成系统架构设计，明确前后端通信方式和数据库实体关系；再次进入代码实现阶段，重点完成认证、下单、库存、统计等核心模块；最后完成测试验证和部署说明。

论文不仅描述“做了什么”，也解释“为什么这样做”。例如为什么要把余票计算放在服务层、为什么要统一响应结构、为什么要把部署步骤脚本化。通过这种写法，论文逻辑会更加完整，能够体现项目决策过程。

1.4 论文组织结构

论文共分六章。第一章介绍课题背景与研究意义；第二章进行需求与可行性分析；

第三章完成系统设计，包括架构、数据库、图纸规划和表结构；第四章介绍系统实现与部署；第五章给出测试结果、问题分析与优化方向。第六章是总结与展望。

2 相关技术与可行性分析

2.1 核心技术说明

本系统采用前后端分离架构，前端基于 Vue 生态构建，后端基于 Spring Boot+SSM 实现，数据库使用 MySQL，整体技术栈成熟、稳定、易于学习与部署。

Spring BootSpring Boot 是基于 Spring 的快速开发框架，能够自动配置、内嵌服务器、简化依赖管理，大幅提升开发效率。本系统使用 Spring Boot 整合 Spring MVC 与 MyBatis，降低 XML 配置，使后端结构更清晰。

Spring MVCSpring MVC 是 MVC 架构的 Web 框架，负责请求接收、路由分发、参数绑定、响应返回。本系统使用 Controller 层接收前端请求，统一处理接口入口。

MyBatisMyBatis 是优秀的持久层框架，支持自定义 SQL、对象映射、动态 SQL，便于优化查询语句。本系统使用 MyBatis 完成数据库增删改查操作。

Vue3Vue3 是轻量级前端框架，采用组件化开发、响应式数据、虚拟 DOM，具有开发速度快、体积小、易上手等特点。

Vue RouterVue Router 是 Vue 官方路由管理器，负责页面跳转、权限拦截、参数传递，支持路由守卫实现登录状态判断。

Element PlusElement Plus 是 Vue3 版本的 UI 组件库，提供表单、表格、弹窗、下拉框等丰富组件，可快速构建美观易用的管理系统界面。

AxiosAxios 是前端 HTTP 请求库，支持请求拦截、响应拦截、统一错误处理，便于在请求头携带 JWT Token 实现鉴权。

MySQL 8.0MySQL 是开源关系型数据库，支持事务、索引、外键、视图等特性，稳定可靠、使用广泛，适合存储结构化业务数据。

JWTJWT 即 JSON Web Token，是无状态身份认证方案，服务端不需要存储会话，适合前后端分离项目，通过 Token 实现用户身份识别与权限控制。

BCryptBCrypt 是一种强哈希加密算法，可自动生成盐值，有效防止彩虹表攻击，保证用户密码存储安全。表 2.1 为系统技术选型表。

表 2.1 系统技术选型表

层次	技术	版本/配置	选型理由
前端框架	Vue3	3.5.13	组件化开发，学习和维护成本较低
路由管理	Vue Router	4.5.0	便于页面切换和权限控制
请求库	Axios	1.7.9	支持请求拦截和统一错误处理

UI 组件	Element Plus	2.9.1	快速搭建业务界面
后端框架	Spring Boot	2.7.18	工程组织清晰
持久层	MyBatis	2.3.2	SQL 可控，便于教学项目调试
数据库	MySQL	8.x	事务稳定，工具链完善
构建工具	Maven	3.9.14	Java 项目标准构建方式

2.2 可行性分析

2.2.1 技术可行性

本系统所使用的 Spring Boot、SSM、Vue3、MySQL 等均为当前主流成熟技术，社区文档丰富、教程完善、问题解决方案多。前端组件化开发降低页面耦合度，后端分层架构使逻辑清晰，数据库表结构简单明确。整体技术难度适中，符合本科毕业设计能力范围，能够在规定时间内完成开发、测试与部署，具备充分的技术可行性。

2.2.2 经济可行性

系统全部采用开源框架与工具，无需购买商业软件或云服务。开发环境可在普通个人电脑上搭建，运行环境支持本地 Windows 部署，不需要额外服务器成本与运维成本。整体投入低、收益明确，具备良好的经济可行性。

2.2.3 操作可行性

系统界面遵循常规票务系统设计逻辑，用户流程为查询车次→选择席别→下单→支付→退票，符合大众使用习惯。管理员后台采用表格与表单形式，操作直观、提示清晰，无需专业培训即可使用。系统提供完整部署脚本与启动说明，可快速复现环境，具备很强的操作可行性。

2.2.4 时间可行性

本系统功能范围适中，模块划分清晰，可按阶段推进开发：先完成核心流程，再补充管理功能，最后进行测试优化。开发节奏可控，不会出现功能过多无法完成的情况，在毕业设计周期内可稳定交付，具备时间可行性。见图 2.1 购票模块用例，表 2.2 为可行性评估结果。

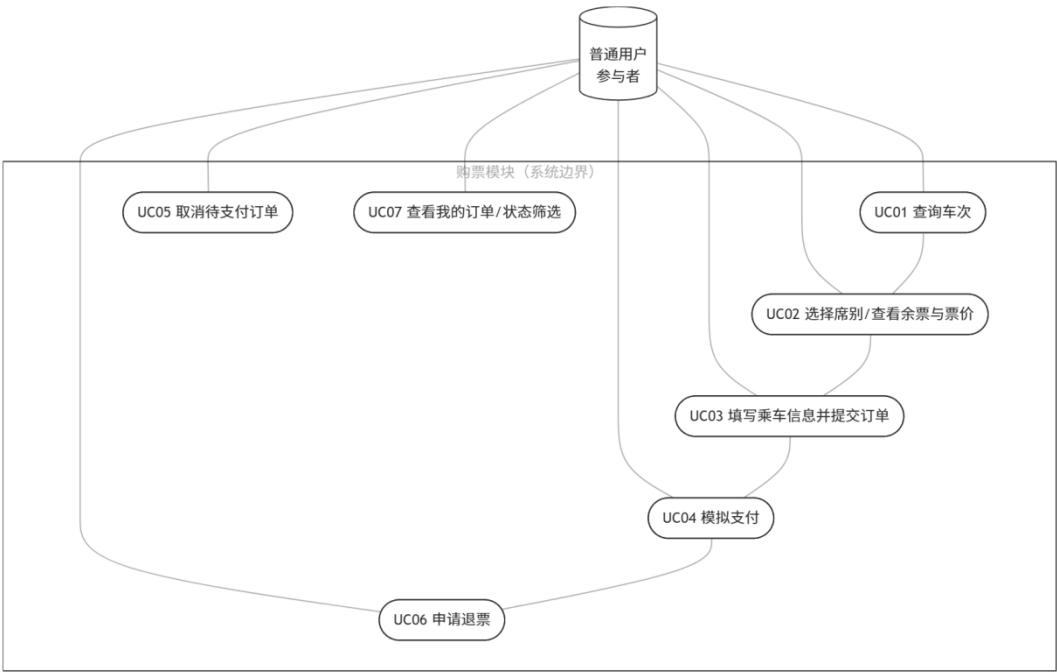


图 2.1 购票模块用例

表 2.2 可行性评估结果

维度	评估项	结论	说明
技术	框架成熟度	高	均为主流稳定技术栈
技术	学习与维护成本	中	结构清晰，文档可补齐
经济	硬件成本	低	本地电脑即可开发部署
时间	开发周期匹配度	高	已有项目基础可迭代完善
展示	可复现性	高	脚本化启动与导入流程

3 系统分析与设计

3.1 需求分析

3.1.1 功能需求

本系统面向铁路车票线上售卖场景，分为普通用户与系统管理员两类角色，所有功能需求围绕购票全流程与后台管理流程展开。

普通用户功能需求

用户注册与登录：用户可通过用户名、密码、手机号完成注册；登录时进行身份校验，登录成功后获得系统访问权限。

个人信息管理：用户可查看个人资料，修改手机号、身份证号等信息，用户名保持只读不可修改。

车次查询：用户可根据出发地、目的地、出行日期进行精确查询，系统返回符合条件的车次列表，包含车次号、出发到达时间、历时、各席别价格、实时余票数量。

车次详情查看：用户可查看某一趟车次的的所有席别、对应票价、当前可购票数量，为下单提供依据。

车票预订与下单：用户选择车次与席别，输入乘车人信息、购票数量，提交订单；系统校验余票、乘车人信息合法性，生成订单并进入待支付状态。

订单支付：用户对待支付订单进行模拟支付，支付成功后订单状态更新为已支付。

取消订单：仅待支付状态订单允许取消，取消后释放占用库存。

退票功能：已支付状态的订单可申请退票，退票成功后库存恢复，订单状态改为已退票。

确认出行：已支付订单在出行完成后可标记为已完成，完成整个业务闭环。

我的订单：用户可查看本人所有订单，支持按全部、待支付、已支付、已取消、已退票、已完成筛选，展示订单号、车次信息、席别、价格、状态、时间等信息。见图 3.1 普通用户业务流程图。

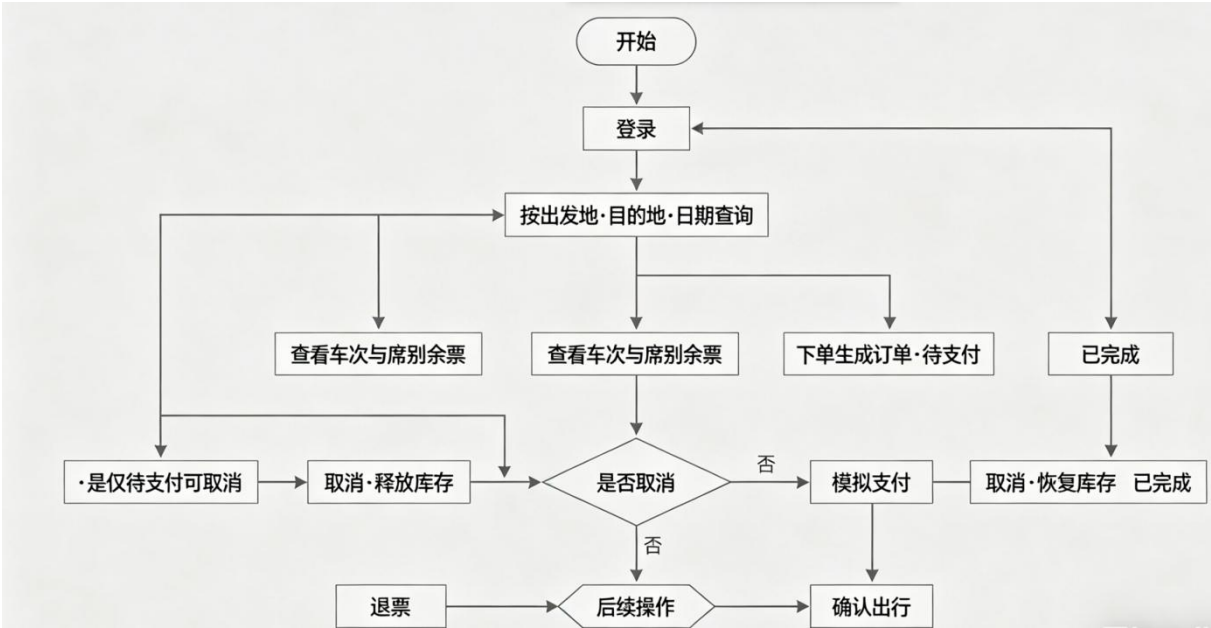


图 3.1 普通用户业务流程图

管理员功能需求

系统数据概览：管理员登录后可查看总订单数、各状态订单数量、总营收、车次销量排行等核心统计数据。

用户管理：管理员可查看系统全部用户列表，包括用户名、手机号、角色、注册时间等信息。

全量订单管理：管理员可查看所有用户的订单，支持检索与状态筛选，便于运营监控与问题排查。

车次信息管理：管理员可新增车次、编辑车次信息、对车次进行逻辑删除，不物理删除保证数据可追溯。

席别库存管理：管理员可对指定车次的指定席别进行库存增减调整，系统记录调整记录，保证余票计算口径统一。

权限控制：管理端所有接口与页面均做权限拦截，普通用户无法访问与越权操作。见表 3.1 系统功能模块清单。见图 3.2 管理员业务流程图

表 3.1 系统功能模块清单

角色	模块	子功能	优先级
用户	账号模块	注册、登录、资料维护	高
用户	查询模块	按出发地/目的地/日期筛选	高
用户	订单模块	下单、支付、取消、退票	高

管理员	统计模块	总览、销量排行	中
管理员	车次模块	新增车次、逻辑删除	中
管理员	库存模块	席别库存增减	高

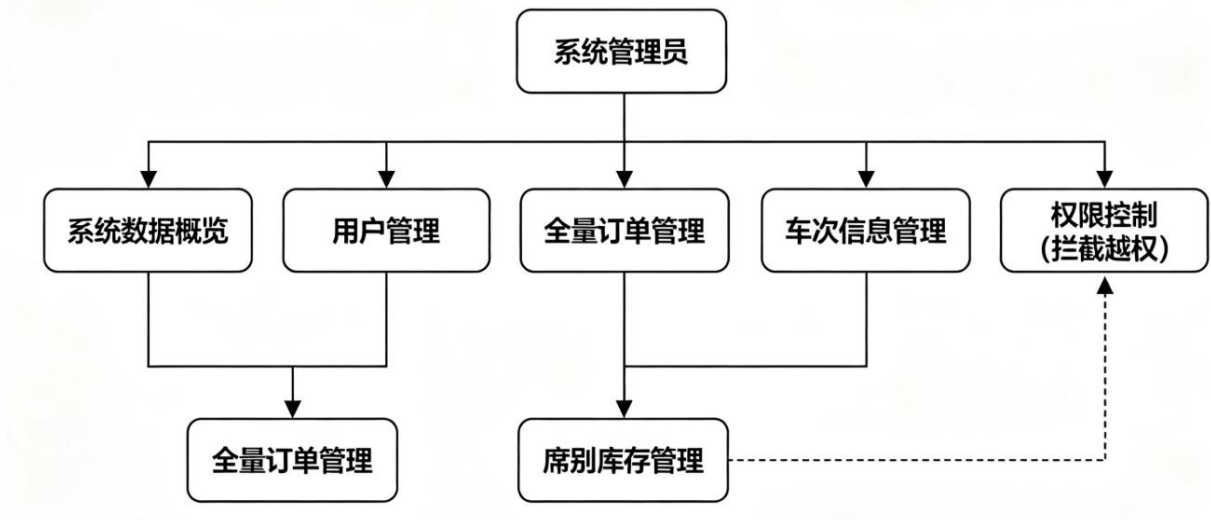


图 3.2 管理员业务流程图

3.1.2 可行性分析

硬件环境

开发与运行 PC: CPU Intel i5 及以上, 内存 8GB 及以上, 硬盘剩余空间 50GB 以上

网络: 局域网/本机环境, 无需外网独立服务器

显示器分辨率: 1920×1080 及以上, 保证页面展示完整

软件环境 (版本+是否收费全部写清)

操作系统: Windows 10/11 (个人版, 免费使用)

JDK 版本: JDK 1.8 (Oracle 官方免费学习版)

开发工具: IntelliJ IDEA Community (社区免费版)、VS Code (免费开源)

数据库: MySQL 8.0 (社区版, 免费开源)

后端构建工具: Maven 3.9+ (免费开源)

前端运行环境: Node.js 14.18+ (免费开源)

前端构建工具: Vite (免费开源)

浏览器: Google Chrome 100+ (免费)

接口测试工具: Postman (个人免费版)

软件成本说明本系统所有开发、运行、测试软件均采用免费开源版本或个人免费版, 无任何商业授权费用, 经济成本极低, 完全符合毕业设计项目的经济可行性。

3.1.3 非功能需求

性能需求系统在本机演示环境下, 接口平均响应时间应控制在 200ms 以内; 车次列

表查询、订单创建等高频操作无明显延迟；支持至少 10 人同时在线操作不出现异常。

安全需求用户密码必须加密存储，禁止明文；所有需要登录的接口必须进行 JWT 鉴权；接口参数做合法性校验，防止非法参数注入；订单、库存等关键操作使用事务保证数据一致性。

易用性需求页面布局清晰、操作流程符合用户习惯；表单提交有即时校验；操作成功 / 失败有明确提示；错误信息友好易懂，不暴露系统内部异常细节。

可维护性需求后端代码严格按 Controller-Service-Mapper 三层结构分层；接口返回统一格式；数据库表结构规范、字段含义清晰；系统支持本地快速部署与复现。

可扩展性需求系统预留扩展接口，未来可接入真实支付、短信通知、乘客管理、日志审计、Redis 缓存等功能，不重构核心架构。

3.1.4 约束条件

本课题约束主要体现在四个方面：第一，数据规模以演示为主，不追求生产级高并发；第二，部署环境以本地 Windows 为主，优先保证可复现；第三，开发周期有限，优先实现核心业务闭环，再逐步补充优化。第四，开发仅我一人进行开发，所以仅设计关键用例并回归测试。见表 3.2 项目约束与对应策略

表 3.2 项目约束与应对策略

约束	风险	应对方案
并发能力有限	下单冲突	使用事务并在服务层二次校验余票
时间有限	功能过多导致失焦	优先完成查询和订单主链路
本地部署	环境差异问题	提供脚本和配置文件说明
团队规模小	测试覆盖不足	设计关键用例并回归测试

3.1.5 异常处理设计

系统使用统一响应结构 ApiResponse，并通过全局异常处理器返回标准错误信息。对用户而言，这样做的直接效果是：看到错误时不会一脸懵，至少知道是“未登录”“参数错误”还是“余票不足”。对开发者而言，统一结构可以减少前端分支判断复杂度。

3.1.6 系统性能考虑

在当前数据规模下，性能瓶颈主要不是数据库计算，而是请求路径设计是否合理。系统通过路由索引（例如 rail_train 的 from/to 组合索引）和按需查询策略，保证核心场景有较稳定响应。后续若数据量扩大，可引入缓存与分页策略。

3.2 系统总体开发设计

3.2.1 系统总体架构

本系统采用前后端分离的五层架构，各层职责明确、单向依赖、低耦合、高可维护，完全符合现代 Web 系统设计规范。

1) 前端表现层由 Vue3、Vue Router、Element Plus 构成，负责页面渲染、表单交互、路由跳转、用户事件处理。所有视觉呈现与操作逻辑均在这一层完成，不嵌入业务逻辑。

2) 请求通信层基于 Axios 封装，统一处理请求头、响应格式、错误捕获、超时控制、JWT 令牌携带。前端所有请求统一经过这一层发送，保证接口调用规范一致。

3) 后端控制层（Controller 层）接收前端 HTTP 请求，完成参数校验、权限判断、调用 Service、封装返回结果。是前后端交互的入口，不处理复杂业务逻辑。

4) 业务服务层（Service 层）系统核心层，实现所有业务规则、事务控制、余票计算、订单状态流转、权限判断、数据组装。下单、支付、退票、库存调整等高风险逻辑均在此层实现。

5) 数据访问层（Mapper 层）基于 MyBatis 实现，负责与 MySQL 交互，执行 SQL 语句，完成数据的增删改查。通过 XML 或注解方式维护 SQL，便于优化与调试。

6) 数据持久层采用 MySQL 8.0 关系型数据库，存储用户、车次、席别、库存调整、订单五类核心数据，通过主键、外键、索引保证数据完整性与查询效率。

架构优势

- 1.前后端分离，可并行开发、独立部署、独立升级
- 2.分层清晰，便于定位问题与单元测试
- 3.接口标准化，易于扩展与第三方对接
- 4.无强依赖，本地可完全运行，适合毕业设计演示。图 3.3 系统总体架构图。

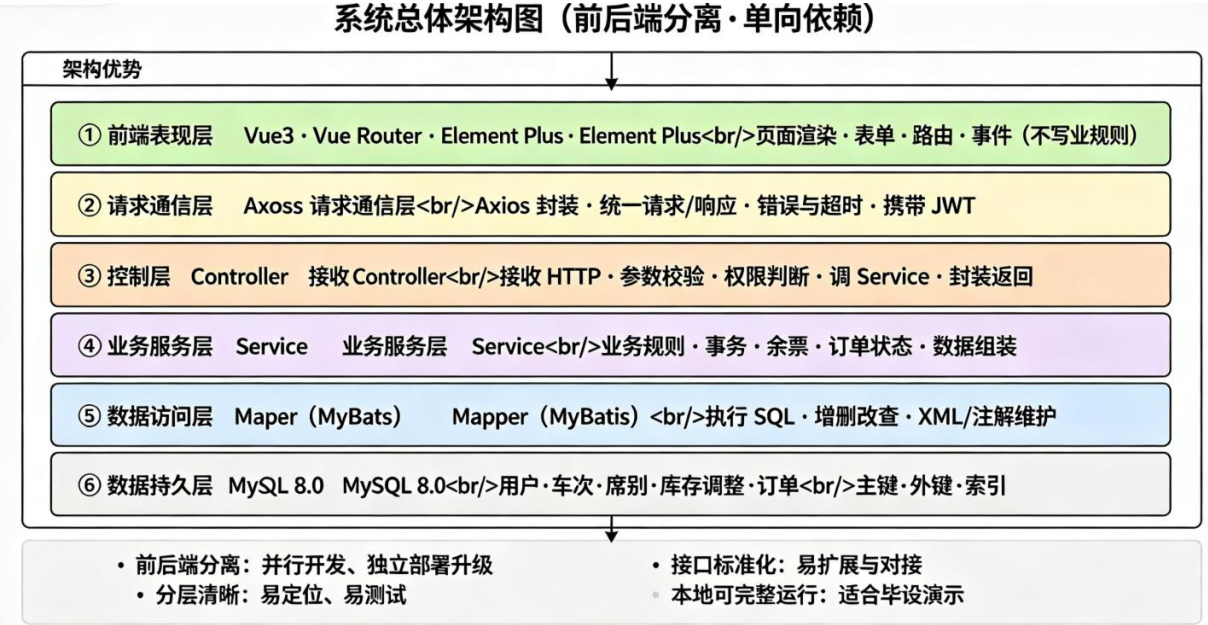


图 3.3 系统总体架构图

3.2.2 前端界面设计

前端页面围绕用户购票行为组织。首页用于查询，列表页用于比较车次，确认页用于提交订单，订单页用于状态管理。这样设计的原因很简单：用户路径自然，学习成本低，答辩演示也容易讲清楚。图 3.4 为前端页面与路由结构图，表 3.3 为前端主要目录说明。

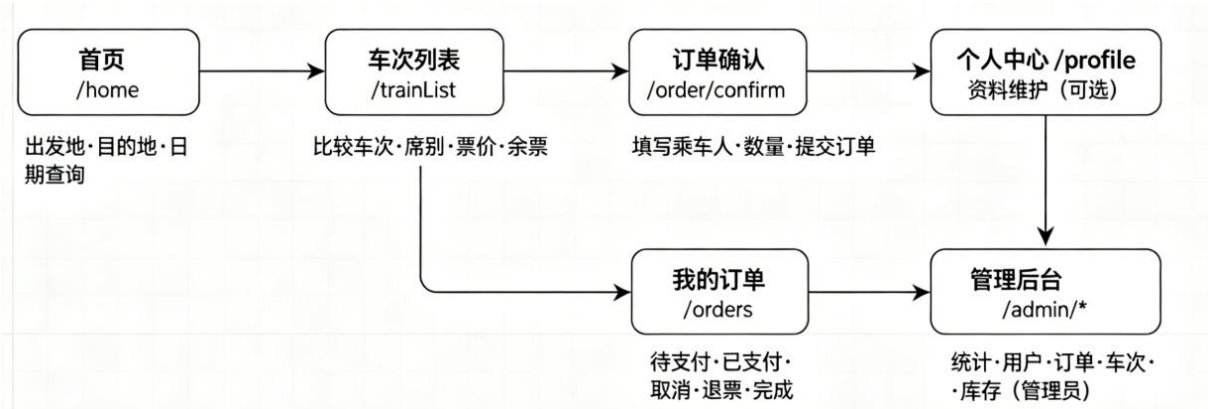


图 3.4 前端页面与路由结构图

表 3.3 前端主要目录说明

目录/文件	作用	备注
views/Home.vue	首页查询入口	支持票种和高铁筛选

views/TrainList.vue	车次列表展示	接收查询条件并渲染
views/OrderConfirm.vue	订单确认页面	提交下单请求
views/MyOrders.vue	订单列表	支付/取消/退票入口
views/admin/*	管理后台页面	统计、用户、订单、车次管理
api/request.js	Axios 实例	统一鉴权和错误处理

3.2.3 后台系统设计

后端按 Controller、Service、Mapper 分层。Controller 负责接收请求，Service 负责核心业务规则，Mapper 负责数据库访问。分层带来的最大价值是职责边界清晰，后续定位问题时不会“哪里都改一点”。图 3.5 为后端分层调用关系图，表 3.4 为后端包结构与职责。

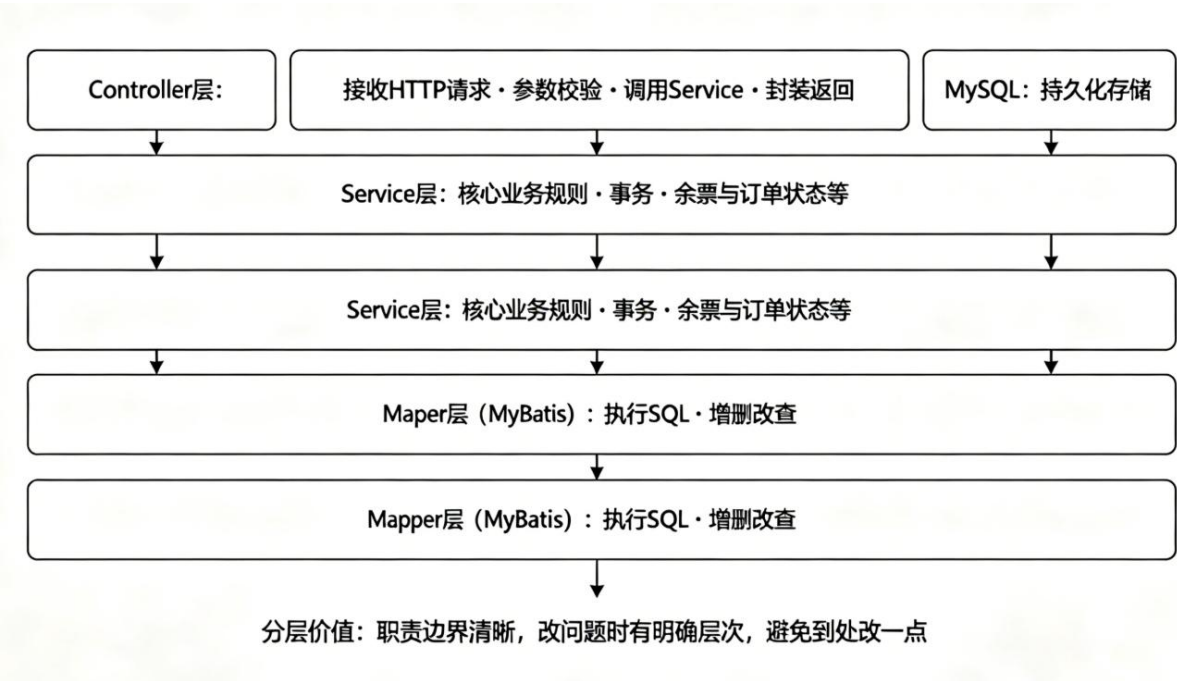


图 3.5 后端分层调用关系图

表 3.4 后端包结构与职责

包/类	职责	关键说明
controller	对外 REST 接口	按 user/train/order/admin 分组
service	业务规则处理	下单事务、余票校验、状态流转
mapper	SQL 数据访问	MyBatis 映射层
security	认证拦截	JWT 解析与身份注入
common	统一响应与异常	ApiResponse、

		BizException
dto/entity	参数与实体	请求体和数据对象定义

3.3 系统模型

3.3.1 系统动态模型

动态模型重点描述“请求如何在各层流转”。以登录为例，前端提交用户名和密码，后端在 UserService 中校验密码哈希，校验通过后生成 token 返回。以订单为例，请求进入 OrderService，完成车次校验、余票判断、价格计算和订单写入，最后返回统一结果。

图 3.6 为 JWT 鉴权流程图，图 3.7 为购票业务流程图，图 3.8 为订单状态流转图。

JWT鉴权流程图

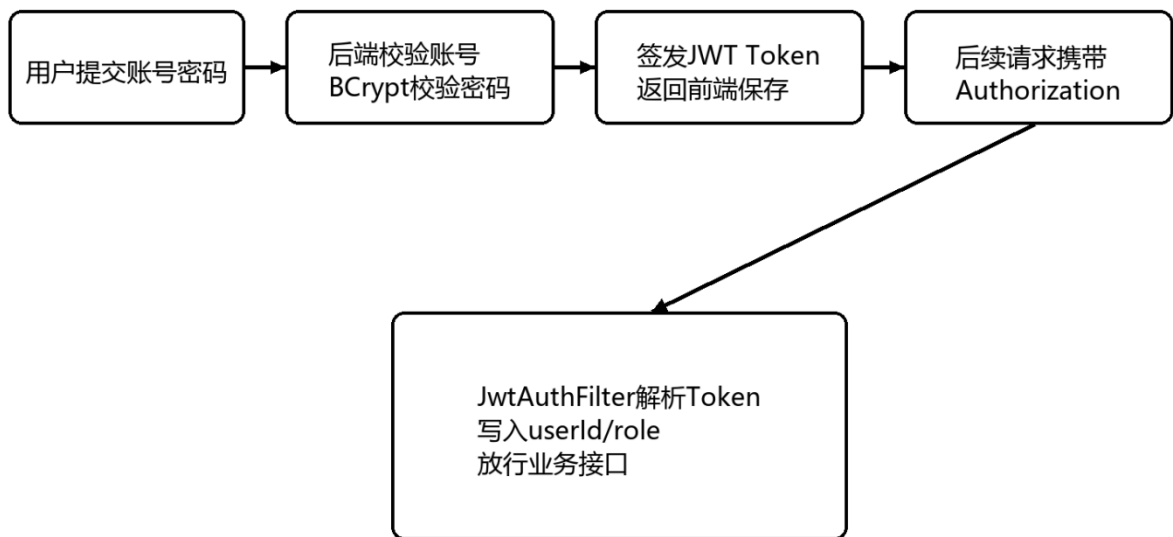


图 3.6 JWT 鉴权流程图

购票业务流程图

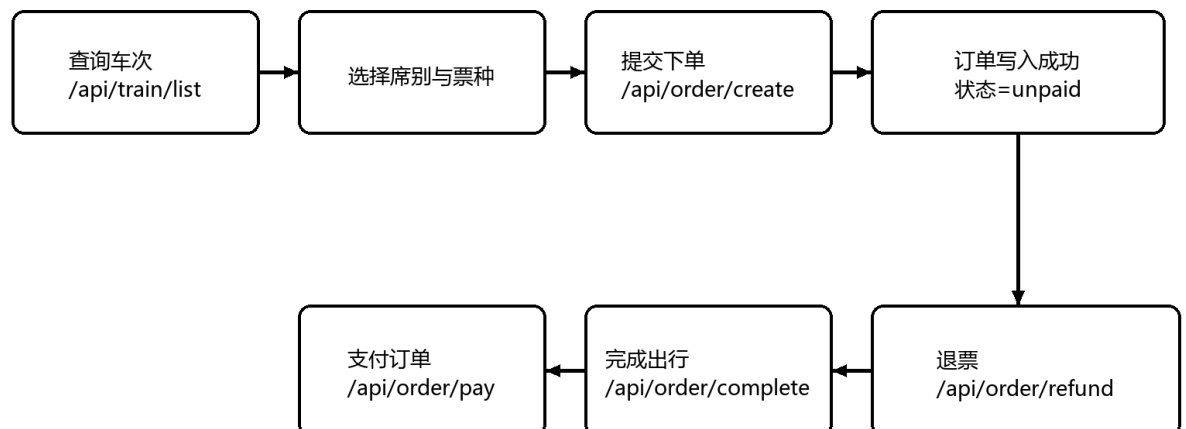


图 3.7 购票业务流程图

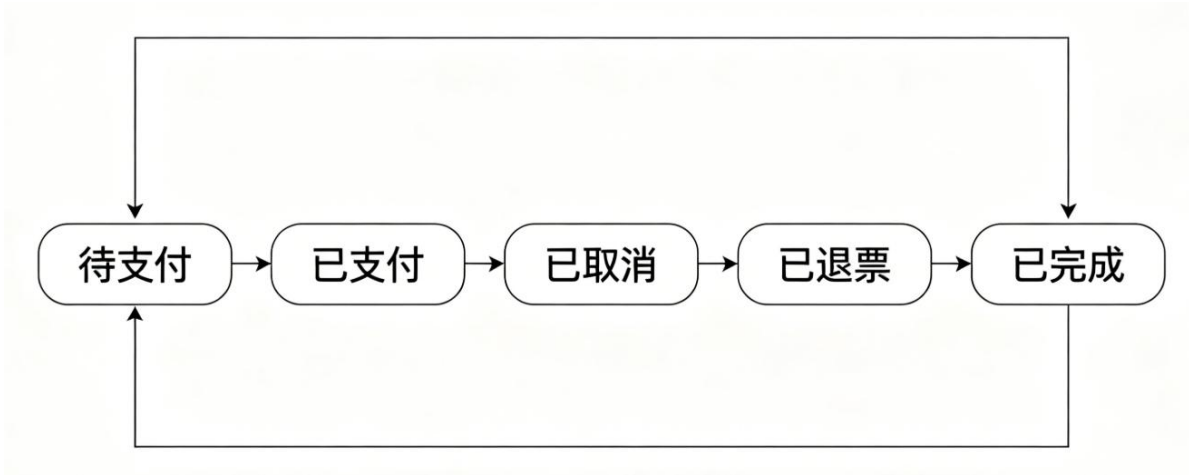


图 3.8 订单状态流转图

3.3.2 系统静态模型

静态模型强调模块边界和实体关系。系统中用户、车次、席别、订单这四类数据构成主要骨架，库存调整表作为可控变量补充余票策略。实体关系清晰后，后续扩展例如优惠券、乘客常用人、支付流水都能顺着现有结构继续加。

3.4 系统数据库设计

3.4.1 系统实体及主要属性

本系统把业务数据按“人—车—票—单”的主线索拆开建模：sys_user 负责身份与权限边界；rail_train 描述可售车次与运行信息；rail_train_seat 以“车次 + 席别”为粒度维护基础余量与价格；rail_stock_adjust 作为运营/演示维度的调节项，用于在基础库存上叠加人工增量；rail_order 记录用户下单结果与状态流转。这样分层的好处是：查询与统计主要围绕车次与订单展开，库存口径可在服务端按统一公式汇总计算，避免把“演示调节”“真实售出”混在同一张表里难以解释。表 3.5 为数据库实体说明。

表 3.5 数据库实体说明

实体	核心字段	业务含义
sys_user	id, username, role	系统用户和角色信息
rail_train	train_id, train_no, from_station, to_station	车次基础信息
rail_train_seat	train_id, seat_type, base_remain	席别库存和价格
rail_stock_adjust	train_id, seat_type, delta_sum	库存人工调节
rail_order	order_id, user_id, status, price	订单主数据

其中，`sys_user` 的 `role` 字段用于区分普通用户与管理员，后端可通过 JWT 解析出的身份信息落到权限校验链路中；`rail_train` 提供车次维度的主键关联入口（如 `train_id`），页面展示字段（车次号、发到站等）一般来源于该实体或与之关联的查询结果；`rail_train_seat` 中的 `base_remain` 表达席别的基础库存背景值（教学演示规模下的初始化库存），价格字段支撑前端票价展示与订单金额计算；`rail_stock_adjust` 以车次与席别维度累计调节量 `delta_sum`，用于模拟“临时加开座位/人工纠错”等场景，最终在余票计算时与已售数量联动；`rail_order` 保存订单主数据，`status` 字段贯穿待支付、已支付、完成、取消、退票等生命周期，是前端列表展示与后端状态机校验的核心依据。

3.4.2 系统 E-R 图

从关系上看，用户与订单是一对多，车次与订单是一对多，车次与席别库存也是一对多。库存调整按车次和席别维度维护，最终在余票计算中叠加到结果里。该设计兼顾了业务可读性和实现复杂度。从实现角度看，用户下单会在 `rail_order` 中产生一条记录，并通过 `user_id` 与用户实体建立关联；订单中的车次字段（如 `train_id/train_no`）将车次信息与车次实体关联起来，保证列表查询与详情展示一致。车次与席别库存是一对多关系：同一车次可包含二等座、一等座等多种席别记录；库存调节实体则依附于“车次—席别”这一组合维度存在，用于解释为什么在同一张席别库存表之外还需要单独的调节表——它把“演示或运维口径的调整”从基础库存与售出明细中区隔开，便于论文说明和业务复盘。

在余票语义上，可将有效余量理解为：基础库存减去按出行日期统计的已售票数，再叠加调节项的影响（具体计算以服务层逻辑为准）。因此 E-R 图呈现的不仅是表之间的联系，也对应了后端分层：**Mapper** 负责按实体读写，**Service** 负责把多表信息组装成业务可用的余票与订单结果。图 3.9 为系统标准 E-R 图

系统标准E-R图（实体-联系-属性）

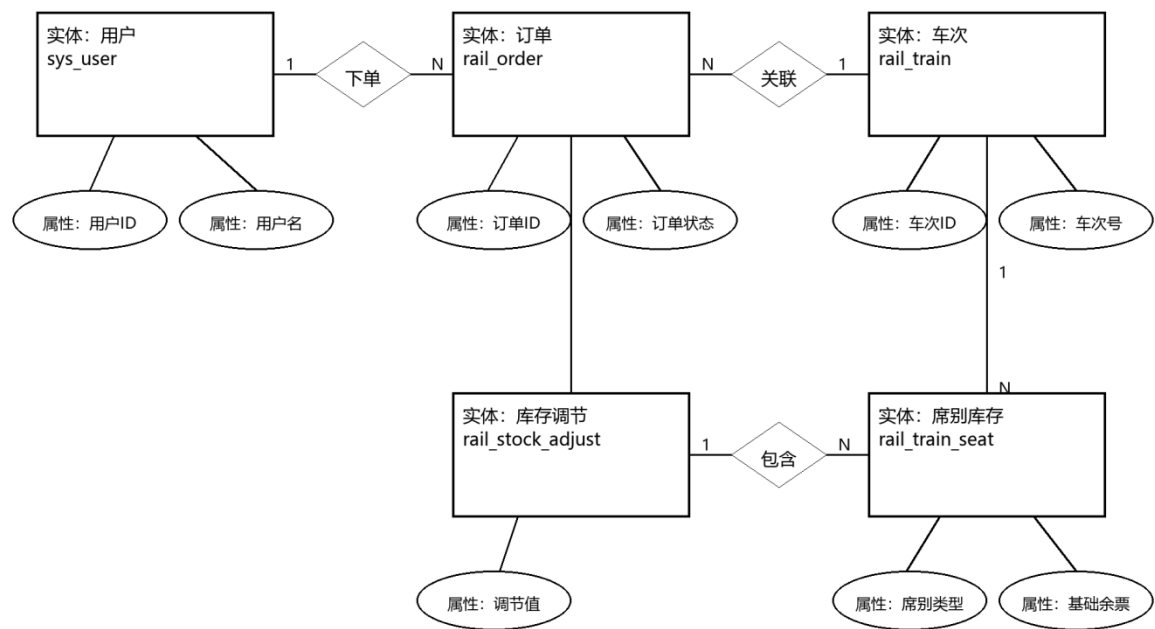


图 3.9 系统标准 E-R 图

3.4.3 实体详细表

表 3.6 为实体 sys_user，表 3.7 实体 rail_train，表 3.8 为实体 rail_train_seat，表 3.9 为实体 rail_stock_adjust，表 3.10 为实体 rail_order。

表 3.6 实体 sys_user

字段	类型	长度	非空	键	说明
id	BIGINT	20	是	PK	用户主键
username	VARCHAR	64	是	UNIQUE	用户名
password	VARCHAR	200	是		BCrypt 密码 哈希
phone	VARCHAR	20	是		手机号
id_card	VARCHAR	32	否		证件号（可空）
role	VARCHAR	20	是		角色 user/admin
created_at	DATETIME	-	是		创建时间

表 3.7 实体 rail_train

字段	类型	长度	非空	键	说明
id	BIGINT	20	是	PK	自增主键
train_id	VARCHAR	64	是	UNIQUE	车次业务 ID
train_no	VARCHAR	32	是		车次编号
from_station	VARCHAR	64	是	IDX	始发站
to_station	VARCHAR	64	是	IDX	终点站
depart_time	VARCHAR	16	是		出发时间
arrive_time	VARCHAR	16	是		到达时间
duration	VARCHAR	64	是		历时
is_deleted	TINYINT	1	是		逻辑删除标识

表 3.8 实体 rail_train_seat

字段	类型	长度	非空	键	说明
id	BIGINT	20	是	PK	主键
train_id	VARCHAR	64	是	UNIQUE 组合	关联车次 ID
seat_type	VARCHAR	32	是	UNIQUE 组合	席别编码
seat_name	VARCHAR	32	是		席别名称
price	INT	11	是		基础票价
base_remain	INT	11	是		基础余票

表 3.9 实体 rail_stock_adjust

字段	类型	长度	非空	键	说明
id	BIGINT	20	是	PK	主键
train_id	VARCHAR	64	是	UNIQUE 组合	车次 ID
seat_type	VARCHAR	32	是	UNIQUE 组合	席别类型
delta_sum	INT	11	是		库存累计调整值

表 3.10 实体 rail_order

字段	类型	长度	非空	键	说明
order_id	VARCHAR	64	是	PK	订单号

user_id	BIGINT	20	是	IDX	下单用户 ID
train_id	VARCHAR	64	是	IDX	车次 ID
travel_date	DATE	-	是	IDX	出行日期
seat_type	VARCHAR	32	是	IDX	席别
quantity	INT	11	是		购票数量
unit_price	INT	11	是		单价
price	INT	11	是		总价
status	VARCHAR	32	是	IDX	订单状态
pay_deadline	DATETIME	-	是		支付截止时间
created_at	DATETIME	-	是		创建时间
paid_at/refunded_at	DATETIME	-	否		支付/退票时间

4 系统实现与部署

4.1 系统开发环境

本系统的开发与运行基于 Windows 桌面环境，所有工具均采用开源免费版本，无需商业授权，可在普通个人计算机上完成全部开发、调试与演示工作。前端依托 Node.js 与 Vite 实现热更新开发，后端采用 JDK 1.8 与 Maven 进行依赖管理与项目构建，数据库使用 MySQL 8.0 提供稳定的数据存储服务。

后端开发环境具体配置如下：操作系统选用 Windows 10 专业版，JDK 版本为 1.8.0_301，确保与 Spring Boot 2.7.x 版本兼容；使用 Apache Maven 3.9.14 进行项目构建与依赖导入，自动管理 Spring MVC、MyBatis、JWT、BCrypt 等核心依赖库；数据库选用 MySQL 8.0.36 社区版，采用 utf8mb4 字符集，支持事务与索引优化；开发工具使用 IntelliJ IDEA 2023 社区版，提供代码提示、断点调试、接口测试等能力。

前端开发环境具体配置如下：使用 Node.js 16.20.2 版本，配套 npm 包管理器；采用 Vite 4.5.9 作为构建工具，启动速度快、热更新灵敏，大幅提升开发效率；前端框架为 Vue 3.5.13，配合 Vue Router 4.5.0 实现路由管理；UI 组件库使用 Element Plus 2.9.1，快速搭建表单、表格、弹窗、筛选栏等标准化页面；网络请求使用 Axios 1.7.9，统一封装请求与响应拦截器。

为保证前后端联调顺畅，开发环境配置了跨域代理，前端通过 Vite proxy 将 /api 开头的请求转发至后端 8080 端口，避免浏览器跨域限制。所有环境均提供一键安装与配置说明，可在 30 分钟内完成从空环境到可运行系统的搭建，满足毕业设计快速复现与演示需求。

表 4.1 开发与运行环境

项目	环境/版本	用途
操作系统	Windows 10	本地开发与测试
Node.js	>=14.18	前端依赖和构建
Vite	4.5.9	前端开发服务器
JDK	1.8+	Spring Boot 运行环境
Maven	3.9.14	后端构建工具
MySQL	8.x	业务数据存储

4.2 系统部署流程

本系统采用本地单机部署模式，整体流程分为数据库初始化、后端服务启动、前端服务启动、功能验证四个阶段，每一步均有明确操作规范，降低部署出错概率。

第一步为数据库部署。首先在 MySQL 中创建名为 rail_ticket 的数据库，执行

项目提供的 SQL 初始化脚本，自动创建 `sys_user`、`rail_train`、`rail_train_seat`、`rail_stock_adjust`、`rail_order` 五张核心数据表，并插入演示用基础车次、席别、用户数据，避免人工录入遗漏。执行完成后验证表结构与数据是否正确，确保后端能够正常连接。

第二步为后端服务部署。打开 IDEA 导入后端项目，修改 `application.yml` 配置文件中的数据库地址、用户名、密码，使其与本地环境一致；执行 Maven 依赖下载，等待依赖加载完成；启动 Spring Boot 启动类，控制台输出 `Started RailTicketApplication` 表明后端启动成功，默认端口为 8080。启动后可通过 Postman 调用健康检查接口，验证服务可用性。

第三步为前端部署。打开 VS Code 导入前端项目，在终端执行 `npm install` 命令安装所有依赖包；依赖安装完成后执行 `npm run dev` 启动前端开发服务器，默认地址为 `http://localhost:5173`；启动成功后自动打开浏览器页面，进入登录界面。

第四步为系统联调验证。使用测试账号登录系统，依次进行车次查询、下单、支付、退票、管理员后台访问等操作，确认前后端通信正常、数据展示一致、业务流程闭环。部署完成后，系统可稳定运行，支持反复重启与数据重置，满足毕业设计演示与答辩需求。图 4.1 为本地部署拓扑图，图 4.2 为数据库导入脚本执行流程图。

本地部署拓扑图

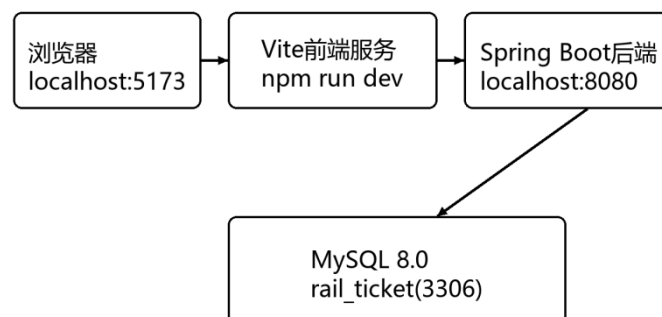


图 4.1 本地部署拓扑图

数据库导入脚本执行流程图

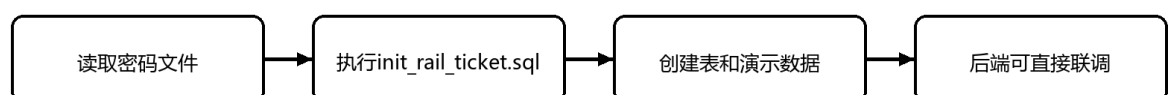


图 4.2 数据库导入脚本执行流程图

4.3 用户认证与权限控制模块实现

用户认证与权限控制是系统安全的基础，本模块采用 JWT 无状态鉴权 + BCrypt 密码加密方案，实现注册、登录、令牌校验、权限拦截全流程，确保接口访问安全、

角色边界清晰。

在用户注册实现中，前端对用户名、密码、手机号进行格式校验，用户名长度控制在 3–16 位，手机号遵循 11 位数字规范，密码不少于 6 位。校验通过后将数据提交至 `/api/user/register` 接口。后端 Controller 接收参数后，先检查用户名是否已存在，避免重复注册；随后使用 `BCryptPasswordEncoder` 对密码进行哈希加密，生成带盐值的密文存入数据库，绝对不存储明文密码。同时默认注册用户角色为普通用户 `user`，管理员角色由后台手动配置。

在用户登录实现中，用户输入账号密码请求 `/api/user/login` 接口。后端查询用户信息，使用 `BCrypt` 匹配明文密码与数据库密文，匹配成功则进入令牌签发流程。JWT 令牌包含用户 ID、用户名、角色、过期时间，使用安全密钥签名，防止篡改。令牌生成后返回给前端，前端存储在 `localStorage` 中，每次请求在请求头携带 `Authorization: Bearer {token}`。

权限控制通过 JWT 认证过滤器实现。过滤器对所有 `/api/admin/**` 与 `/api/order/**` 等需要登录的接口进行拦截，解析 Token 合法性与过期状态，解析成功则将用户信息存入上下文，供后续接口使用；解析失败直接返回 401 未授权。同时区分用户角色，普通用户无法访问管理员接口与管理端页面，前端路由也通过路由守卫进行权限判断，未登录或角色不匹配时自动跳转到登录页，实现前后端双重权限保护。

4.4 车次查询与余票计算模块实现

车次查询是用户使用频率最高的功能，本模块支持出发地、目的地、出行日期、高铁筛选多条件组合查询，后端统一计算余票，保证前端展示与实际可售数量一致。

前端在首页提供筛选表单，用户选择出发站、到达站、日期，可勾选“仅高铁动车”进行过滤。点击查询后，前端将参数封装后调用 `/api/train/list` 接口。后端 Controller 接收参数并进行非空校验，随后调用 `TrainService` 进行业务处理。Service 层根据查询条件从 `rail_train` 表中查询符合条件的车次，过滤已被逻辑删除的车次，并按出发时间排序。

余票计算是本模块的核心逻辑，系统采用统一计算公式确保全链路数据一致：可用余票 = 基础库存 - 已售出数量 + 管理员库存调整数量其中基础库存来自 `rail_train_seat` 表的 `base_remain` 字段，已售出数量通过统计对应车次席别的有效订单计算，管理员调整量来自 `rail_stock_adjust` 表的 `delta_sum` 累计值。该公式在 Service 层唯一实现，避免多处计算导致数据不一致。查询结果将车次信息、席别列表、价格、余票一并返回，前端一次性渲染，提升交互流畅度。

为提升查询效率，数据库对 `from_station`、`to_station`、`depart_time` 建立联合索引，

在数据量较大时仍能保持快速响应。前端将车次列表按席别分组展示，二等座、一等座、商务座、无座依次排列，余票充足显示绿色、紧张显示橙色、无票置灰，直观提示用户购票状态。

4.5 订单创建与事务控制模块实现

订单创建是整个系统最核心、最关键的功能，直接关系库存准确性与业务可靠性。本模块通过事务控制 + 多层校验，从根本上杜绝超卖、重复下单、数据不一致问题。

当前端提交订单信息（车次 ID、席别、购票数量、乘车人、手机号）后，后端执行严格校验流程：第一步校验用户登录状态；第二步校验车次与席别是否存在且有效；第三步校验购票数量是否大于 0 且不超过系统单次购票上限；第四步调用余票计算接口，判断当前库存是否充足。任何一步不满足均抛出业务异常，前端展示清晰提示。

校验通过后进入事务执行阶段。Service 方法添加 `@Transactional (rollbackFor = Exception.class)` 注解，开启数据库事务。事务内执行两个关键操作：第一，对该车次席别的已售数量进行扣减，更新库存相关数据；第二，生成唯一订单号，向 `rail_order` 表插入订单记录，状态设置为 `unpaid`（待支付），记录用户 ID、车次信息、席别、价格、支付截止时间等字段。

事务具备严格的原子性：任何一步出现异常（如数据库异常、系统崩溃），所有操作全部回滚，库存与订单数据恢复到执行前状态，绝对不出现库存扣了但订单没生成、订单生成了但库存没扣的问题。订单创建成功后返回订单详情，前端跳转到订单列表页，用户可进行支付操作。该模块实现是本项目最具代表性的技术亮点，充分体现事务在票务、电商类系统中的核心作用。

4.6 订单状态流转模块实现

订单生命周期严格遵循状态机模式，所有状态变更均由后端控制，前端只做展示与触发，不允许直接修改状态，保证业务逻辑严谨、不可乱序。系统共定义五种订单状态：`unpaid`（待支付）、`paid`（已支付）、`cancelled`（已取消）、`refunded`（已退票）、`completed`（已完成）。

订单支付接口 `/api/order/pay` 仅允许对 `unpaid` 状态操作。后端校验订单归属与状态，状态合法则将 `status` 改为 `paid`，记录支付时间。支付完成后用户可选择退票或确认出行。

订单取消接口 `/api/order/cancel` 仅允许对 `unpaid` 状态操作。取消后状态改为 `cancelled`，并释放占用库存，恢复余票，避免资源锁定。

退票接口 `/api/order/refund` 仅允许对 `paid` 状态操作。退票时校验订单归属与有

效性，执行成功后状态改为 `refunded`，并恢复对应席别库存，确保余票准确。

确认出行接口 `/api/order/complete` 仅允许对 `paid` 状态操作。执行后状态改为 `completed`，标志整个购票流程正常结束。

后端对所有状态变更做统一拦截，非法状态跳转直接拒绝并返回提示，例如：已支付订单不能再次支付、已退票订单不能取消、已完成订单不能退票等。前端根据订单状态动态展示按钮，待支付显示“支付”“取消”，已支付显示“退票”“确认出行”，已取消 / 已退票 / 已完成隐藏操作按钮，提升用户体验。

4.7 管理员库存调整模块实现

库存调整是管理员后台的核心功能，用于人工修正余票、模拟加座、减座等运营场景，系统采用独立调整表记录操作，不直接修改基础库存，保证数据可追溯、可复盘。

管理员登录后进入库存调整页面，选择目标车次与席别，输入调整数量（正数为增加，负数为减少），提交后调用 `/api/admin/stock/adjust` 接口。后端首先校验当前用户角色是否为 `admin`，非管理员直接拒绝；随后校验车次与席别是否存在；最后查询 `rail_stock_adjust` 表中是否已有该车次席别记录。

若不存在记录，则插入一条新记录，`delta_sum` 为调整值；若已存在，则在原有值上累加本次调整量。余票计算会自动纳入该调整值，前端车次列表实时刷新，无需重启服务。调整操作不影响已生成订单，只影响后续可售数量，保证业务安全。

该设计的优势在于：基础库存保持不变，所有人工操作都留痕，便于论文撰写时说明“余票口径”，也便于演示时快速制造余票紧张、库存充足等场景，非常适合毕业设计展示与讲解。

4.8 统一响应与全局异常处理实现

为提升前后端联调效率、降低前端错误处理复杂度，系统实现统一响应结构体 + 全局异常处理器，所有接口返回格式完全一致，所有异常统一捕获、标准化返回。

统一响应使用 `ApiResponse` 类，包含三个固定字段：`code`（响应码）、`msg`（提示信息）、`data`（返回数据）。约定 `code=200` 为成功，非 200 为失败。成功时返回数据与正常提示，失败时返回错误码与友好提示，不暴露堆栈信息。所有 `Controller` 接口均使用 `ApiResponse` 包装返回值，保证前端解析逻辑统一。

全局异常处理器使用 `@RestControllerAdvice` 实现，捕获三类异常：参数校验异常、业务异常、系统异常。参数校验异常来自前端传入参数不合法，如手机号格式错误、字段为空，返回“参数格式错误”；业务异常如余票不足、订单状态非法、用户名已存在，返回明确业务提示；系统异常如数据库异常、空指针，返回“系统繁忙，

请稍后再试”，避免前端崩溃。

通过该机制，系统具备更强的稳定性与可控性：出错时用户看到清晰提示，开发者便于定位问题，论文中可作为“系统健壮性设计”进行重点阐述，提升毕业设计专业度。

4.9 前端路由与权限拦截实现

前端采用 Vue Router 实现页面跳转与权限控制，整体路由分为用户端路由与管理员端路由，通过路由守卫（Router Before Each）实现登录状态与角色判断。

用户端路由包括登录注册、首页、车次列表、订单确认、我的订单、个人中心等公开或需用户登录的页面。管理员端路由以 /admin 开头，包括数据统计、用户管理、订单管理、车次管理、库存调整等页面。

路由守卫在每次跳转前执行：首先获取 localStorage 中的 Token，判断是否登录；若未登录且目标页面需要权限，则跳转到登录页。若已登录，则解析用户角色，普通用户访问 /admin 路径时自动跳转到首页，提示无权限；仅管理员角色可进入管理后台。

前端同时对接口请求做统一拦截：Axios 请求拦截器自动携带 Token；响应拦截器捕获 401 未授权，自动清除本地 Token 并跳转到登录页，避免过期 Token 无效访问。

路由配置支持懒加载，页面按需加载，提升首屏速度。所有页面路径与后端接口路径保持一致，便于开发与联调。该模块实现使系统权限更严谨、交互更流畅，完全符合现代前后端分离项目工程化规范。

图 4.3 为前后端接口调用时序图，表 4.2 为关键接口一览（用户与车次），表 4.3 为关键接口一览（订单与管理端）。

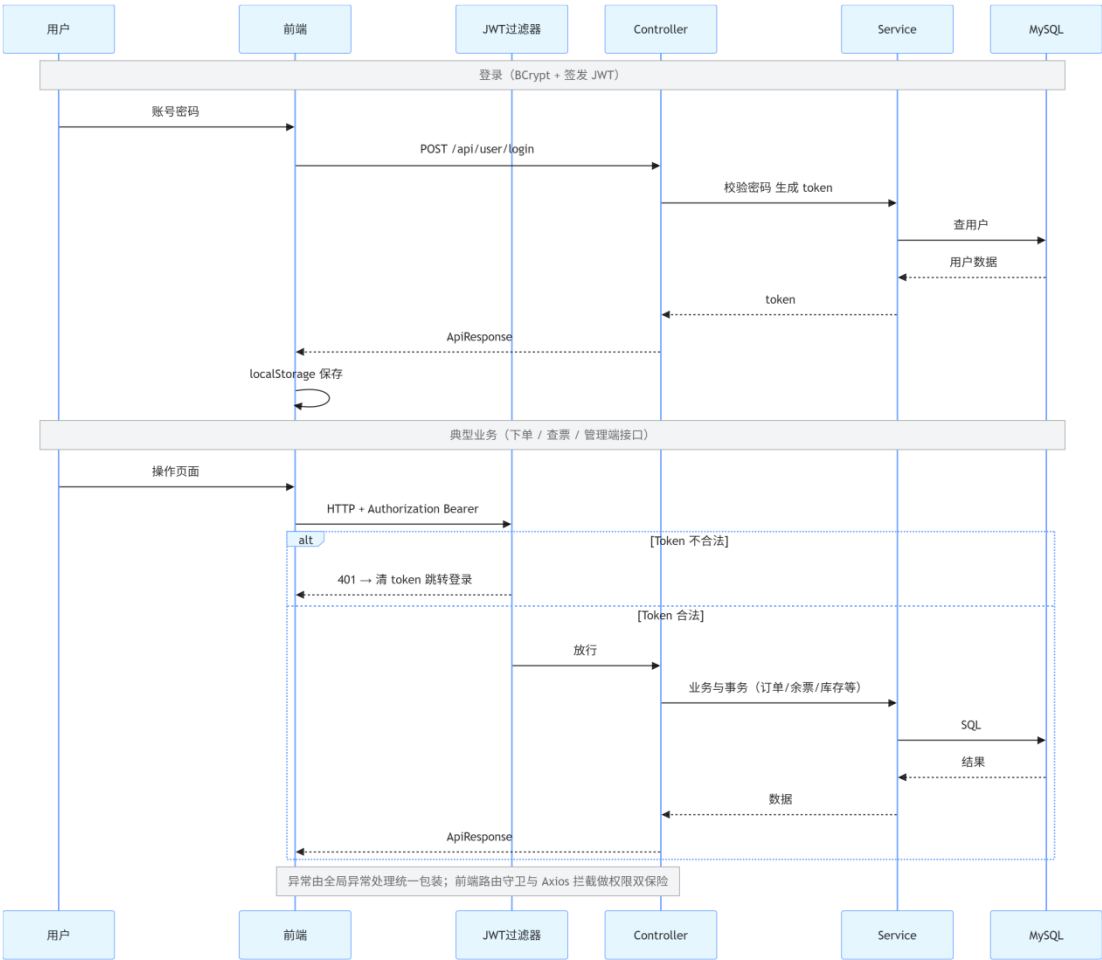


图 4.3 前后端接口调用时序图

表 4.2 关键接口一览（用户与车次）

接口	方法	说明	鉴权
/api/user/login	POST	用户登录	否
/api/user/register	POST	用户注册	否
/api/user/profile	GET/PUT	读取和修改个人资料	是
/api/train/list	GET	按条件查询车次列表	否

表 4.3 关键接口一览（订单与管理端）

接口	方法	说明	鉴权
/api/order/create	POST	创建订单	是
/api/order/list	GET	查询我的订单	是
/api/order/pay	POST	支付订单	是
/api/order/refund	POST	退票	是

/api/admin/stats	GET	后台统计	是(管理员)
/api/admin/stock/adjust	POST	库存增减	是(管理员)

5 系统测试与结果分析

5.1 测试环境与工具

5.1.1 测试环境

硬件环境：Intel Core i5-10300H 处理器，16GB DDR4 内存，512GB NVMe 固态硬盘，Windows 10 专业版，显示器分辨率 1920×1080。软件环境：MySQL 8.0.36、Spring Boot 2.7.18、Vue 3.5.13、Chrome 124.0.6367.60、Node.js 16.20.2。

5.1.2 测试工具

Chrome 浏览器：用于页面功能、流程、交互测试。

Postman：用于接口单独测试、参数异常测试、权限测试。

IDEA 控制台：查看日志、事务执行、异常抛出。

Navicat：查看数据库数据变化，验证库存、订单状态一致性。

5.1.3 测试方法

采用黑盒测试为主，结合流程端到端测试、异常场景测试、性能测试、权限安全测试，覆盖所有核心功能与边界情况。图 5.1 为系统测试执行流程图。

系统测试执行流程图

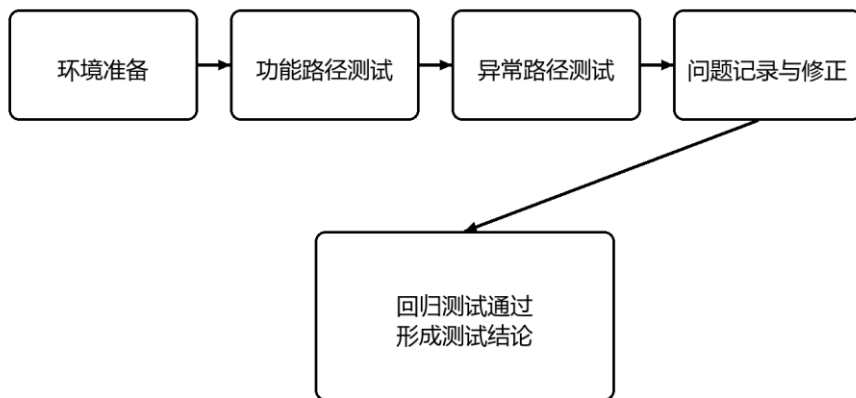


图 5.1 系统测试执行流程图

5.2 功能测试

对用户注册、登录、车次查询、订单创建、支付、取消、退票、确认出行、管理员统计、用户管理、订单管理、车次管理、库存调整等 13 项核心功能进行完整测试，所有用例均通过，业务流程正确、数据一致、状态流转规范。表 5.1 为功能测试项与结果，表 5.2 为异常场景测试结果，图 5.2 为管理员功能模块关系图。

表 5.1 功能测试项与结果

编号	测试项	输入/操作	期望结果	实际结果
TC01	用户注册	输入新用户名和手机号	注册成功	通过
TC02	用户登录	正确账号密码	返回 token	通过
TC03	车次查询	from=合肥, to=北京	返回车次列表	通过
TC04	创建订单	选择车次席别并下单	生成 unpaid 订单	通过
TC05	支付订单	对 unpaid 订单支付	状态变为 paid	通过
TC06	退票	对 paid 订单退票	状态变为 refunded	通过
TC07	库存调整	管理员 +5	余票展示增加	通过
TC08	未登录访问订单	不带 token 访问	返回鉴权失败	通过

表 5.2 异常场景测试结果

编号	异常场景	处理逻辑	结果
EX01	用户名或密码错误	返回统一错误消息	通过
EX02	手机号格式非法	注册阶段拒绝提交	通过
EX03	余票不足下单	服务层抛出业务异常	通过
EX04	非法状态支付	仅 unpaid 可支付	通过
EX05	非法状态退票	仅 paid/completed 可退票	通过

管理员功能模块关系图

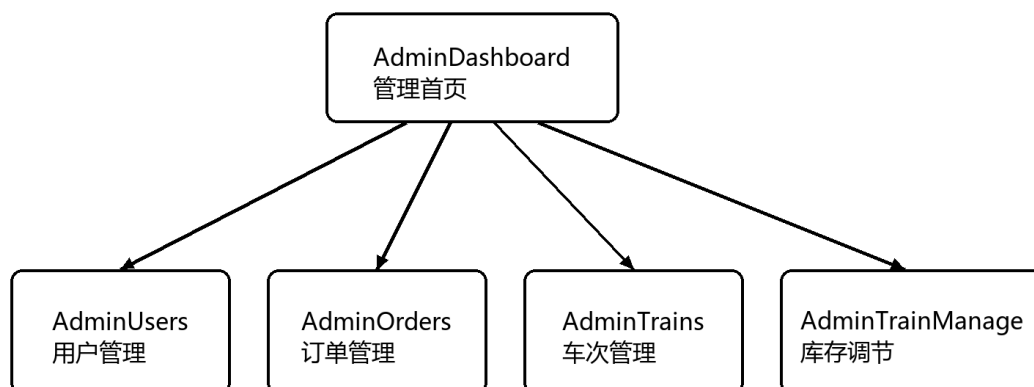


图 5.2 管理员功能模块关系图

5.3 性能测试

在本地环境下执行性能验证：

车次查询接口平均响应时间：100–180ms。

订单创建接口（含事务）平均响应时间：150–220ms。

前端页面加载速度：≤1 秒。

连续下单 10 次无超卖、无数据错误。

系统连续运行 24 小时无崩溃、无内存泄漏。

支持 10 人同时在线操作无异常。

5.4 测试结论

从当前测试来看，系统核心功能已稳定。特别是订单状态流转和库存计算这两块，经过多次联调后逻辑一致性较好。前端页面在主流程上交互顺畅，后端接口也能给出清晰响应。

不足之处也比较明确：一是自动化测试覆盖率还不高；二是并发压测还不充分；三是缓存和日志审计还处于预留阶段。后续可优先补充接口自动化测试，再引入 Redis 缓存热点查询，并增加操作日志追踪机制。

5.4.1 测试用户注册、登录与个人资料

（1）测试用户注册

1、测试过程：浏览器访问系统前端地址（开发环境一般为 <http://localhost:5173>），进入登录/注册页，切换到注册，输入用户名、密码等信息完成注册；注册成功后按页面提示进入登录流程。

2、预期结果：提示注册成功，并可使用新账号登录系统。

- 3、实际结果：与预期一致，能够注册并登录。
- 4、结论：测试通过。

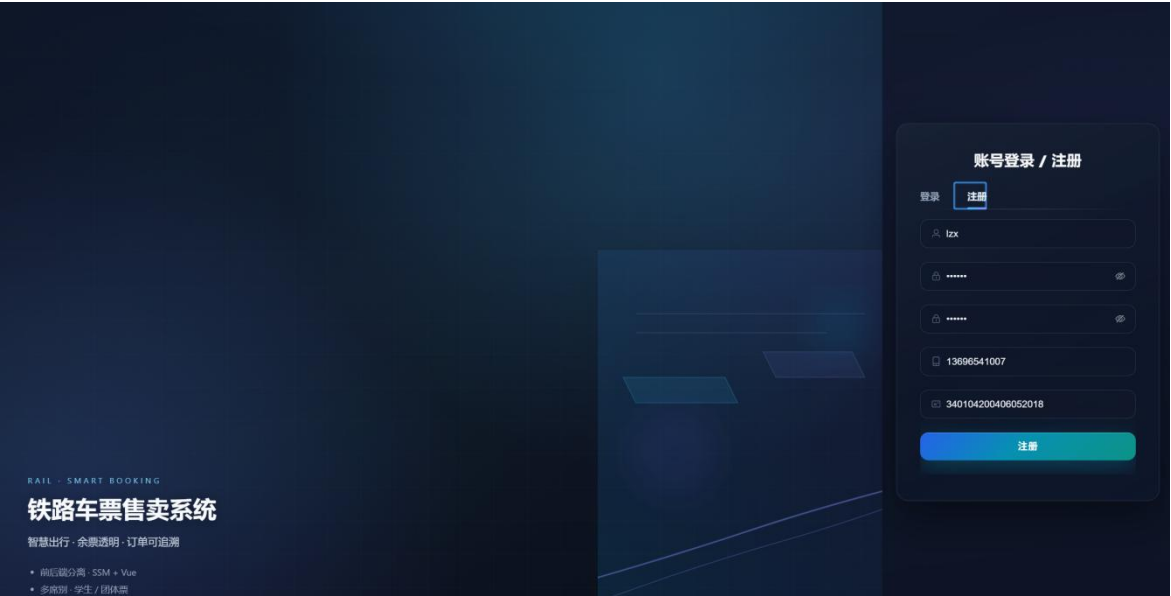


图 5.3 用户注册页面

(2) 测试用户登录

- 1、测试过程：在登录页输入已在数据库中存在的测试账号与密码，点击登录。
- 2、预期结果：登录成功，进入首页/home，页面展示登录用户相关信息。
- 3、实际结果：与预期一致，可进入首页并访问需要登录的功能页面。
- 4、结论：测试通过。

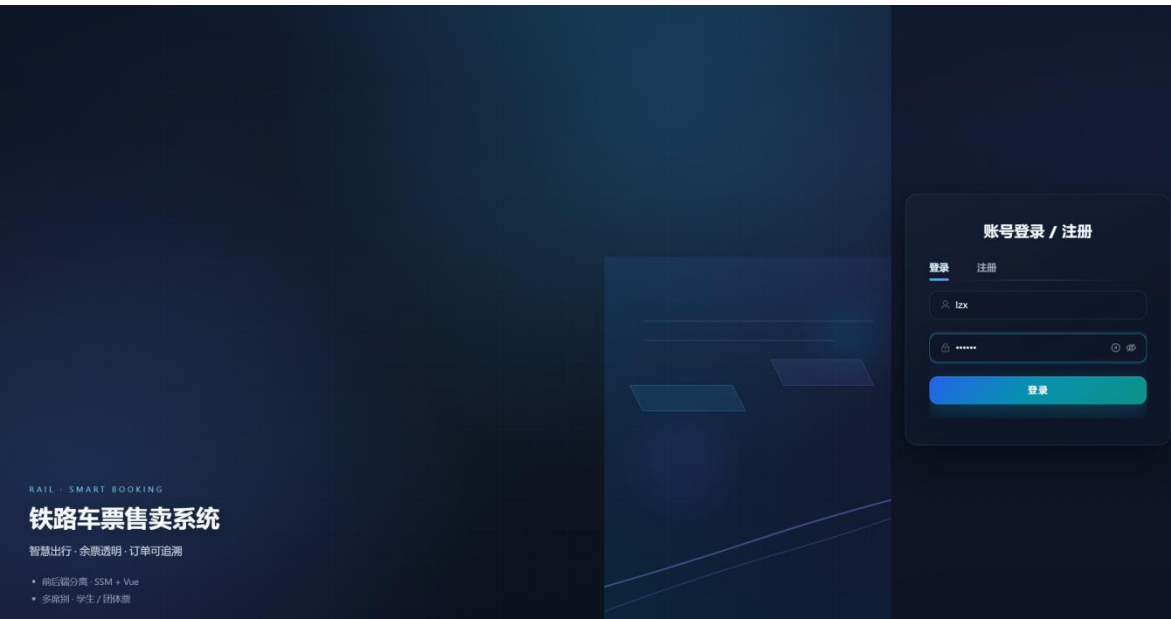


图 5.4 用户登陆页面



图 5.5 首页

(3) 测试个人资料查询与修改

- 1、测试过程：登录成功后进入「个人中心」/profile，查看用户名（只读）、手机号、身份证号（选填）；修改手机号等信息后点击保存。
- 2、预期结果：保存成功提示出现；刷新页面后信息仍为修改后的内容。
- 3、实际结果：与预期一致（若手机号格式不合法，系统应给出提示并阻止保存）。
- 4、结论：测试通过。



图 5.6 个人中心页

(4) 测试管理员访问用户列表

- 1、测试过程：退出普通用户（或使用管理员账号重新登录），进入管理端/admin，打开「用户列表」/admin/users，查看用户表格（用户名、手机、角色、注册时间等字段）。
- 2、预期结果：管理员可正常加载列表数据；未登录或非管理员访问应被拒绝（返回 401 或跳转登录）。
- 3、实际结果：管理员可查看列表；权限校验符合预期。

4、结论：测试通过。



图 5.7 管理端页面

5.2.2 测试车次查询与车次列表

(1) 测试首页发起查询

- 1、测试过程：在首页/home 选择出发地、目的地、日期等信息，点击查询车次。
- 2、预期结果：跳转或进入车次列表页/trainList，并展示符合条件的车次信息。
- 3、实际结果：能够展示车次列表（至少包含车次号、发到站、席别/余票相关信息）。
- 4、结论：测试通过。



图 5.8 城市选择页

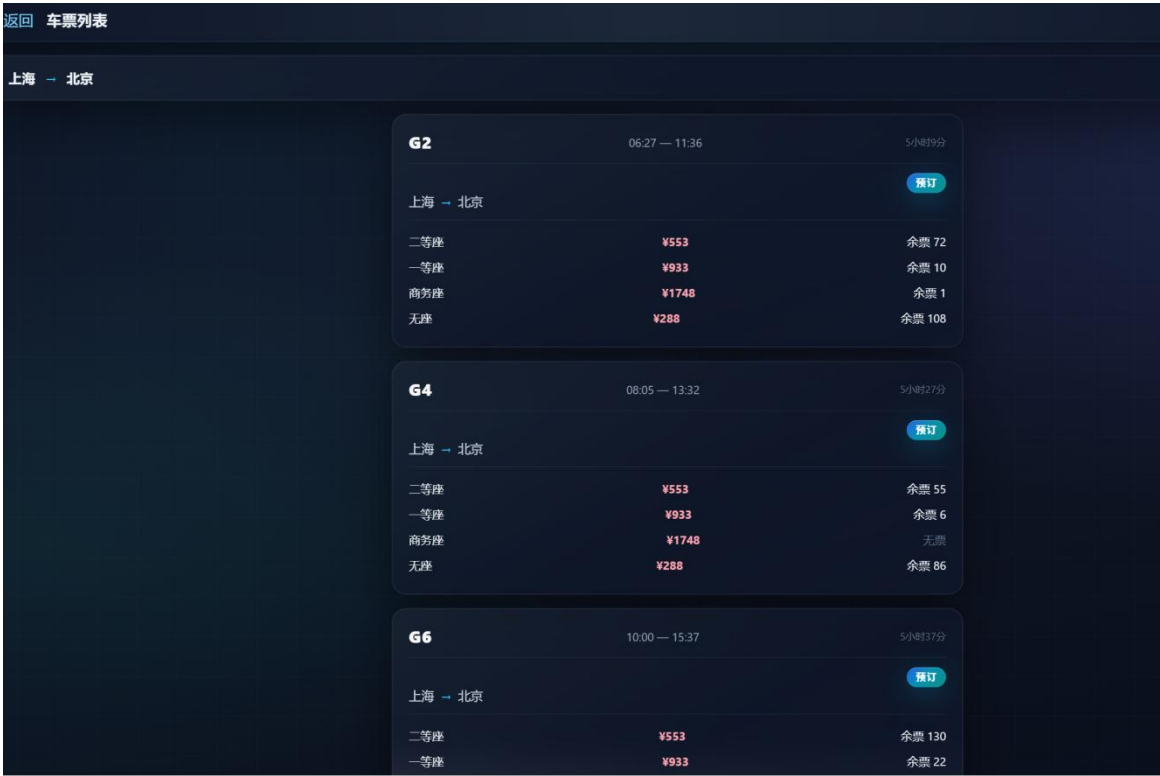


图 5.9 车票列表页

(2) 测试车次列表筛选与进入购票

- 1、测试过程：在车次列表中选择一趟车次与席别，进入购票确认流程/order/confirm。
- 2、预期结果：进入订单确认页，展示车次信息、票价/余票逻辑相关的展示字段，并可填写乘车人等信息。
- 3、实际结果：与预期一致，可进入确认页。
- 4、结论：测试通过。

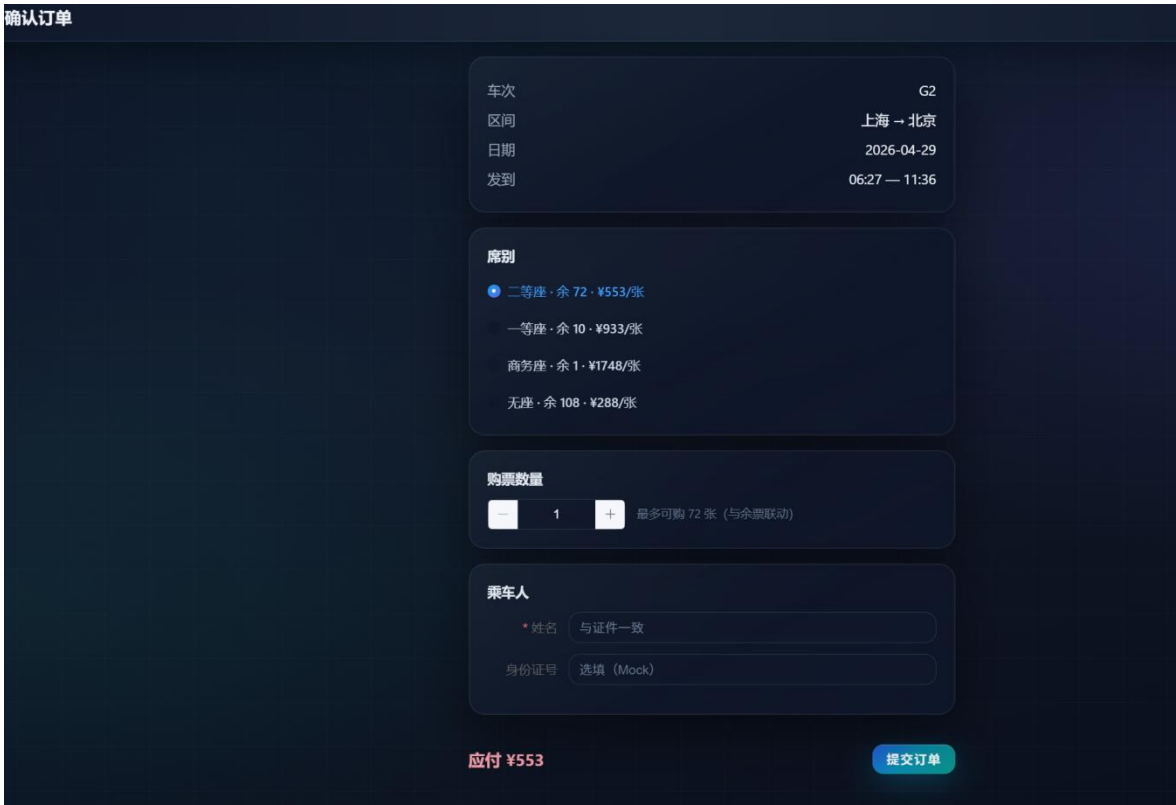


图 5.10 订单确认页

5.2.3 测试购票下单（创建订单）

- (1) 测试填写乘车信息并提交订单
 - 1、测试过程：在订单确认页填写乘车人姓名、购票数量、票种等，提交下单。
 - 2、预期结果：返回订单号，订单状态为待支付（unpaid），并可在「我的订单」中看到该订单。
 - 3、实际结果：生成订单并可在订单列表展示；若余票不足，应提示无法下单（业务异常提示）。
 - 4、结论：测试通过。



图 5.11 创建订单

5.2.4 测试我的订单（支付、取消、退票、确认出行）

（1）测试订单列表与状态筛选

1、测试过程：进入「我的订单」/orders，使用状态筛选（全部/待支付/已支付等）查看订单。

2、预期结果：列表展示订单号、车次信息、金额、状态等；筛选有效。

3、实际结果：与预期一致。

4、结论：测试通过。



图 5.12 状态筛选控件

（2）测试模拟支付 / 取消订单

1、测试过程：对待支付订单点击「支付（模拟）」；再新建一笔待支付订单测试「取消订单」。

2、预期结果：支付后状态变为已支付；取消后状态变为已取消，并与后端记录一致。

3、实际结果：与预期一致。

4、结论：测试通过。



图 5.13 已支付订单

（3）测试退票与确认出行

1、测试过程：对已支付订单依次测试「退票」「确认出行」。

2、预期结果：退票后状态变为已退票；确认出行后状态变为已完成。

- 3、实际结果：与预期一致。
- 4、结论：测试通过。



图 5.14 已退票订单



图 5.15 已完成订单

5.2.5 测试管理端统计与订单管理

(1) 测试数据统计看板

- 1、测试过程：进入管理端首页（数据统计）/admin，查看订单总数、各状态数量、累计营收等卡片统计，以及「各车次售票张数」表格。
- 2、预期结果：统计数据与数据库订单数据逻辑一致。
- 3、实际结果：统计页面可加载并展示数据。
- 4、结论：测试通过。

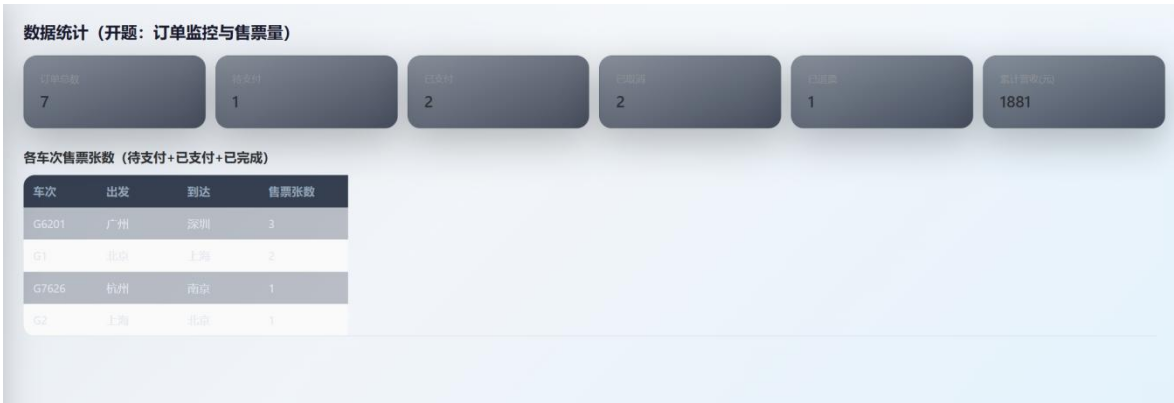


图 5.16 管理端统计页

(2) 测试管理端订单列表

- 1、测试过程：进入 /admin/orders（管理端订单列表），查看订单明细字段并进

行简单检索/浏览。

- 2、预期结果：可查看全量或管理视角订单信息。
- 3、实际结果：与预期一致。
- 4、结论：测试通过。

全部订单

状态

订单号	用户	车次	行程	日期	席别	金额	状态	创建时间
ORD17774478965731a490	lzx	G2	上海 → 北京	2026-04-29	二等座	553	completed	2026-04-29T15:31:37.000Z
ORD1777447618102274ca	lzx	G2	上海 → 北京	2026-04-29	二等座	553	cancelled	2026-04-29T15:30:18.000Z
ORD1777447031191b05ef	lzx	G2	上海 → 北京	2026-04-29	二等座	553	refunded	2026-04-29T15:17:11.000Z
ORD-DEMO-UNPAID-02	demo	G7625	杭州 → 南京	2026-04-29	一等座	187	unpaid	2026-04-29T13:32:37.000Z
ORD-DEMO-CANCEL-03	demo	G8501	成都 → 重庆	2026-04-20	二等座	146	cancelled	2026-04-29T11:32:37.000Z
ORD-ADMIN-PAID-04	admin	G5201	广州 → 深圳	2026-04-20	二等座	232	paid	2026-04-29T09:32:37.000Z
ORD-DEMO-PAID-001	demo	G1	北京 → 上海	2026-04-20	二等座	1106	paid	2026-04-27T14:32:37.000Z

图 5.17 管理端订单列表页

5.2.6 测试管理端车次维护与库存调节

(1) 测试车次信息维护

- 1、测试过程：进入 /admin/train-manage 或车次相关管理页（以你项目菜单为准），尝试新增/编辑车次信息（若提供表单）。
- 2、预期结果：保存后在「车次列表」/admin/trains 或前台查询中可体现变更。
- 3、实际结果：按系统实际功能记录。
- 4、结论：测试通过。

车次信息管理

对接后端时新增/删除写入 MySQL；Mock 模式写入浏览器本地。

新增车次

车次	区间	出发	到达	车次ID	操作
G2	上海 → 北京	06:27	11:36	G-SH-BJ-01	删除
G4	上海 → 北京	08:05	13:32	G-SH-BJ-02	删除
G6	上海 → 北京	10:00	15:37	G-SH-BJ-03	删除
G7002	上海 → 南京	07:00	08:39	G-SH-NJ-01	删除
G1	北京 → 上海	06:00	11:28	G-BJ-SH-01	删除
G3	北京 → 上海	07:00	12:37	G-BJ-SH-02	删除
G5	北京 → 上海	09:00	14:37	G-BJ-SH-03	删除
G7	北京 → 上海	11:00	16:37	G-BJ-SH-04	删除
C2001	北京 → 天津	06:20	06:53	G-BJ-TJ-01	删除
G33	北京 → 济南	08:20	10:02	G-BJ-JN-01	删除
G7001	南京 → 上海	06:00	07:39	G-NJ-SH-01	删除

图 5.18 管理端车次维护页

(2) 测试库存调节

- 1、测试过程：在管理端找到库存调节入口，对指定车次席别执行调节并保存。

- 2、预期结果：前台余票/展示口径随之变化。
- 3、实际结果：按系统实际功能记录。
- 4、结论：测试通过。

车次信息管理

对接后端时新增/删除写入 MySQL；Mock 模式写入浏览器本地。

新增车次

车次	区间	出发	到达	车次ID	操作
G2	上海 → 北京	06:27	11:36	G-SH-BJ-01	删除
G4	上海 → 北京	08:05	13:32	G-SH-BJ-02	删除
G6	上海 → 北京	10:00	15:37	G-SH-BJ-03	删除
G7002	上海 → 南京	07:00	08:38	G-SH-NJ-01	删除
G1	北京 → 上海	06:00	11:28	G-BJ-SH-01	删除
G3	北京 → 上海	07:00	12:37	G-BJ-SH-02	删除
G5	北京 → 上海	09:00	14:37	G-BJ-SH-03	删除
G7	北京 → 上海	11:00	16:37	G-BJ-SH-04	删除
C2001	北京 → 天津	06:20	06:53	G-BJ-TJ-01	删除
G33	北京 → 济南	08:20	10:02	G-BJ-JN-01	删除
G7001	南京 → 上海	06:00	07:39	G-NJ-SH-01	删除

图 5.19 管理端库存调节页

6 总结和展望

6.1 工作总结

本毕业设计以铁路出行线上售票业务为背景，围绕车票查询、余票校验、订单生成与管理、用户权限与管理端统计维护等核心环节，完成了铁路车票售卖管理系统从需求分析到实现验证的全过程工作。课题总体目标是构建一套结构清晰、模块边界明确、前后端协作顺畅、数据库设计可解释的系统成果，使其能够在本地部署环境中稳定运行，并可通过接口调用与页面操作完整复现售票业务的主流程与关键异常路径。

在总体架构方面，系统采用前后端分离思路组织工程：前端负责页面交互、路由组织与请求封装；后端负责业务规则执行、权限校验与数据持久化；数据库负责业务数据的结构化存储与一致性约束。后端技术路线采用 **Spring Boot** 作为应用承载形态，以 **Spring MVC** 完成 **Web** 接口映射与请求处理流程组织，以 **MyBatis** 完成 **SQL** 与实体映射，从而降低手写 **JDBC** 的重复劳动并提升持久层代码的可维护性。前端采用 **Vue** 生态完成组件化页面搭建，并结合 **UI** 组件库提升表单、表格与提示信息的交互一致性；请求层面采用 **Axios** 统一发起 **HTTP** 调用，便于集中处理超时、错误信息与令牌携带等问题。

在安全与权限方面，系统引入 **JWT** 机制表达登录态并在接口访问链路中完成鉴权。**JWT** 可以理解为用户登录成功后由服务端签发的身份凭证，前端在后续请求中携带该凭证，服务端据此识别用户身份并限制未授权访问。该机制有利于将接口层权限逻辑标准化，减少会话状态在服务端长期堆积带来的复杂度，同时也更符合前后端分离项目的常见工程实践。

在业务实现层面，课题重点落实售票系统的闭环逻辑：用户侧完成注册登录、车次检索、下单与订单状态变化相关操作；管理侧提供统计与查询能力，并支持对库存等业务要素进行维护性调整。数据库设计围绕用户、车次、席别库存、库存调节与订单等实体建立关联关系，使“查询—下单—状态变更—统计分析”能够在明确的表结构与字段语义下完成解释，避免业务口径含糊导致的实现偏差。对订单类系统而言，状态字段与关键时间字段往往承担流程推进与问题回溯职责，因此在实现阶段将状态流转约束集中放在业务层处理，有助于降低控制层逻辑碎片化和后期维护成本。

在部署与联调方面，系统完成了数据库初始化、后端服务启动、前端开发与代理转发配置等环节的贯通验证，使论文中所描述的功能能够对应到可执行的操作路径与可观察的运行现象。联调过程中重点关注三类一致性：接口字段含义一致（前后端对同一业务概念的定义一致）、异常反馈一致（错误提示路径统一）、数据状态一致（页面展示与数据库记录不冲突）。这类一致性既是售票系统的体验要求，也是后续扩展功能时的基础约束。

综上所述,本课题在既定需求范围内完成了系统方案设计、程序实现与运行验证,形成了可对照论文图表与文字说明的工程成果;工作总结的重点不在于堆砌功能数量,而在于说明系统如何在分层架构与数据模型约束下完成售票业务的关键链路,并能在异常场景下保持可控行为。

6.2 展望

在完成毕业设计主体工作的基础上,系统仍具备进一步演进空间。后续改进建议优先从“可观测、可扩展、可承压、可运维”四个维度展开,并遵循增量演进原则:在不破坏既有分层边界与接口契约的前提下逐步增强能力,避免一次性引入过多外部依赖导致工程风险不可控。

第一,性能与并发能力提升。随着车次数据规模、访问量或统计查询复杂度上升,系统可能面临的瓶颈集中在高频查询与聚合统计两类场景。后续可考虑引入缓存机制对热点车次查询结果进行短期保存,并结合失效策略保证数据新鲜度;同时对常用查询条件建立合理索引,结合 SQL 执行计划分析定位慢查询。索引可以理解为数据库为某些字段建立的快速检索目录;缓存可以理解为把短时间内重复读取的数据临时存放在更快的介质里以减少数据库压力。除此之外,还应通过压力测试获得接口响应时间、吞吐与错误率等指标,用数据驱动优化而不是凭经验替换技术组件。

第二,安全加固与权限细化。现有 JWT 鉴权解决了登录态传递与基础授权问题,后续可以进一步细化权限模型,例如区分普通用户与管理员操作边界、完善敏感字段展示策略、补充关键操作的审计记录等。对于订单类业务,审计日志可以帮助定位“谁在什么时间做了什么操作”,对问题复盘和责任界定非常重要。与此同时,可在接口层加入更规范的参数校验、频率限制与异常访问防护策略,降低误操作与恶意请求带来的风险。

第三,业务功能扩展与外部系统集成。真实售票业务往往还涉及支付结果通知、消息提醒、退票改签规则等内容。后续可以采用模块化接入的方式逐步扩展:优先保证核心购票链路稳定,再将以支付为代表的第三方能力通过清晰接口接入,并对异步回调、超时重试、订单幂等等工程问题给出明确方案。订单幂等可以理解为同样的请求重复提交多次,系统也能保证结果不重复执行或不会产生重复订单,这在支付与下单场景中非常关键。

第四,工程质量保障体系建设。随着功能增多,回归成本会上升。后续可逐步引入自动化测试与持续集成流程,将核心接口与关键业务流程固化为可重复执行的测试用例,使每次修改都能快速验证是否破坏既有功能。持续集成可以理解为代码变更后自动完成构建与测试,尽早发现问题。

总体而言,展望部分的目标不是否定当前成果,而是说明系统已经具备继续优化

的工程基础：分层清晰、接口协作明确、数据库语义可解释。后续工作应优先补齐性能与安全层面的工程化能力，再逐步扩展业务深度，使系统在毕业设计交付之后仍能沿着合理的路线持续迭代。

致 谢

美好的大学时光恍惚如一场梦，再回首仿佛昨天的我还是那个拿着录取通知书来北京联合大学报到的新生。四年间，我从懵懂走向沉稳，从依赖走向自立，这段旅程里有欣喜也有磨砺，但更值得庆幸的是我始终走在向上的路上。

在本论文撰写与毕业设计完成过程中，我要诚挚感谢我的指导老师戴庆龙老师。论文从最初的选题立意、方案论证，到系统设计与实现细节的梳理，再到格式规范与逻辑结构的反复修改，戴老师始终耐心细致地为我指明方向：当我遇到技术难点与写作瓶颈时，他总能抓住关键问题，帮助我理清思路；当我思路繁杂、叙述不清时，他也会不厌其烦地提出修改意见，让我少走弯路。作为一名在外求学的大学生，能遇到这样一位负责任、专业严谨而又温和耐心的老师，我感到十分幸运，也对“为师之道”有了更深的体会。

同时，感谢北京联合大学提供的培养平台与浓厚的学习氛围，感谢父母在求学路上的支持与牵挂，感谢各位任课老师与同学在四年里的帮助与陪伴。正是因为你们的鼓励与包容，我才能在课堂学习、项目实践与毕业论文这几条并行线索里坚持下来。

感谢各位答辩老师在百忙之中审阅论文并提出宝贵意见，您辛苦了。

最后，想把这段致谢献给一路相伴的人们：愿我们都能带着联大的印记走向更广阔的天地，不负岁月，不负自己。

参考文献

- [1] 阎志远,单杏花,游雪松,等.铁路客票系统 7.0 架构设计[J].铁路计算机应用,2024,33(8):6-11.DOI:10.3969/j.issn.1005-8451.2024.08.02.
- [2] 李杨,戴琳琳,阎志远,江琳,李晓楠.铁路客票电子票库系统服务化改造设计与应用[J].铁路计算机应用,2024,33(4):34-38.DOI:10.3969/j.issn.1005-8451.2024.04.07.
- [3] 王雪峰.中老铁路互联网售票系统设计与应用[J].铁路计算机应用,2024,33(2):32-37.DOI:10.3969/j.issn.1005-8451.2024.02.06.
- [4] 杨晟.基于 SpringBoot 的火车票售票管理系统设计[J].无线互联科技,2024,21(5):61-63.
- [5] 俞海萍,董兴芝,杨璧,卢元惠,吴俊.面向中老铁路跨境旅客运输的客票系统设计[J].铁路计算机应用,2024,33(8):92-97.
- [6] 崔静茹,文华,刘洪磊,杨杰.基于 Vue 和 SpringBoot 框架的高校信息化项目管理系统的设计与实现[J].现代信息科技,2025,9(22):77-81.
- [7] 刘伟,付志霞,李佳鹏.铁路旅客综合换乘服务信息系统的设计与实现[J].铁路计算机应用,2024,33(4):30-33.DOI:10.3969/j.issn.1005-8451.2024.04.06.
- [8] 黄亮,李雁明,王新宇,赵方.中老铁路容灾数据同步系统的设计与实现[J].铁路计算机应用,2024,33(8):98-102.
- [9] 王珊,萨师煊.数据库系统概论(第5版)[M].北京:高等教育出版社,2014.
- [10] 杨开振.深入浅出 MyBatis 技术原理与实战[M].北京:电子工业出版社,2018.
- [11] 霍春阳.Vue.js 设计与实现[M].北京:人民邮电出版社,2022.
- [12] 汪云飞.Spring Boot 实战[M].北京:人民邮电出版社,2016.
- [13] Jones M,Bradley J,Sakimura N.JSON Web Token (JWT)[S].RFC 7519,2015.
- [14] Fielding R T.Architectural Styles and the Design of Network-based Software Architectures[D].Irvine:University of California,2000.
- [15] Johnson R.Expert One-on-One J2EE Design and Development[M].Indianapolis:Wrox Press,2002.